Chapter 2

Intelligent Styling: Using AI to Master layout styling

Chapter 2 1

Intelligent Styling: Using AI to Master layout styling 1

3.1. introduction 3

3.2. Cascading Style Sheets, CSS 3

Within declaration 4

Where should I place my CSS instructions? 4

3.3. CSS Selectors: Targeting Elements and Understanding Priorities 7

3.3.1. Type Selector (Element Selector) 7

3.4. Text in CSS 10

3.4.1. Text Properties 10

3.4.2. Typeface Terminology 10

3.5. Element appearance and spacing 15

3.5.1. border 15

3.5.2. margin 16

3.5.3. padding 19

3.6. CSS Layout Properties 23

3.6.1. Basic CSS Layout Design 23

3.6.2. Display properties 26

3.6.3. Flexbox container 27

3.7. Navigation Bars 38

3.7.1 Vertical Navigation Bars 38

3.7.2. Horizontal Navigation Bars 39

3.7. Color in CSS 41

3.7.1. Colors 41

3.8. Image manipulation in CSS 46
3.8.1. Position Property 46
3.8.2. Background properties 53
3.8.3. z-index property 59
3.8.4. Filter images 60
3.9. Understand CSS animation 67
3.9.1. CSS transitions 67
3.9.2. CSS transform property 69
3.9.3. @keyframes 72
3.9.4. CSS animation using transition and keyframe 72
3.10. Responsive Webpages 90
3.10.1. @media query 91
3.10.2. Using CSS Grid to Build Modern Layouts and Responsive Web Pages 99
3.11 Building Interactive Web Interfaces with CSS — Slideshows and Modal Windows 107
3.11.1. Creating Slideshow using CSS animation 108
3.11.2. Creating a modal window using HTML and CSS 117
Bibliography **130**

3.1. introduction

Cascading Style Sheets (CSS) have long been the foundation of visual design on the web, transforming plain HTML into engaging, responsive, and professional user interfaces. Yet for many learners and developers, mastering CSS can feel overwhelming. From understanding the box model and layout systems like Flexbox and Grid to managing responsiveness and cross-browser compatibility, CSS requires both technical precision and creative insight.

In today's rapidly evolving technological landscape, Artificial Intelligence (AI) has emerged as a powerful learning companion and development tool. AI can help generate styling suggestions, explain complex layout behaviors, debug design issues, and even recommend best practices for accessibility and performance. Rather than replacing the designer or developer, AI enhances the learning process by offering instant feedback, alternative solutions, and intelligent automation.

This chapter explores how AI can be integrated into the process of learning and applying CSS effectively. You will discover how to use AI tools to accelerate experimentation, deepen your understanding of styling concepts, and refine your design workflows. By combining foundational CSS knowledge with intelligent assistance, you can move beyond trial-and-error coding and begin creating polished, responsive, and maintainable web designs with confidence.

Intelligent styling is not just about writing CSS—it is about learning to think like a designer while leveraging AI as a collaborative partner in mastering the art and science of web presentation.

3.2. Cascading Style Sheets, CSS

CSS (Cascading Style Sheets) allows you to create rules that specify how the content of an element should appear. For example, you can specify that the background of the page is cream, all paragraphs should appear in gray using the Arial typeface, or that all level one headings should be in a blue, italic, Times typeface.

CSS works by associating rules with HTML elements. These rules govern how the content of specified elements should be displayed. A CSS rule contains two parts: a **selector** and a **declaration.**

Selectors indicate which element the rule applies to. The same rule can apply to more than one element if you separate the element names with commas.

Declarations indicate how the elements referred to in the selector should be styled. Declarations are placed between brackets.

Declarations are split into two parts (a property and a value) are separated by a colon.



Within declaration

The declaration should specific the property and value of element of the selector.

Properties indicate the aspects of the element you want to change. For example, color, font, width, height and border. Each declaration ends with a semicolon. You can always refer to w3schools.com to check the list of all properties that is available in CSS.

Values specify the settings you want to use for the chosen properties. For example, if you want to specify a color property then the value is the color you want the text in these elements to be.



Where should I place my CSS instructions?

Externally: Your CSS code can be writing in a text editor and save it with file extension .css
Then the .css can be found using the link element.

Internally: You can also include CSS rules within an HTML page by placing them inside a **<style>** element, which usually sits inside the <head> element of the page.

Using external CSS

The **<link>** element can be used in an HTML document to tell the browser where to find the CSS file used to style the page. It is an empty element (meaning it does not need a closing tag), and it lives inside the <head> element. It should use three attributes:

href: This specifies the path to the CSS file (which is often placed in a folder called css or styles).

type: This attribute specifies the type of document being linked to. The value should be text/css.

rel: This specifies the relationship between the HTML page and the file it is linked to. The value should be stylesheet when linking to a CSS file.

An HTML page can use more than one CSS style sheet. To do this it could have a <link> element for every CSS file it uses. For example, some authors use one CSS file to control the presentation (such as fonts and colors) and a second to control the layout.

Exercise 1) Open an AI platform and create a new project folder named **"Intro to CSS."** In the AI prompt, type the following instruction: *Create a simple application that includes an <h1> heading and a <p> paragraph element. Link an external CSS file named style.css to the project.*

Open **VS Code** and create a new project folder named **Intro_CSS**. Inside this folder, create two files: **index.html** and **style.css**, following this structure:

```
Intro_CSS/
│
├──── index.html
└──── style.css
```

Next, open **index.html** and paste the HTML code generated by the AI prompt into the file. Then, open **style.css** and paste the CSS code generated by the AI prompt into it. Finally, open **index.html** in a web browser to see the applied styles in action.

Using Internal CSS

You can also include CSS rules within an HTML page by placing them inside a **<style>** element, which usually sits inside the <head> element of the page.

The **<style>** element should use the type attribute to indicate that the styles are specified in CSS. The value should be text/ css.

When building a site with more than one page, you should use an external CSS style sheet. This:

- Allows all pages to use the same style rules (rather than repeating them in each page).
- Keeps the content separate from how the page looks.
- Means you can change the styles used across all pages by altering just one file (rather than each individual page).

Exercise 2) Before the closing </body> tag, add a new <h1> element with the class author and insert your name like this:

```
<h1 class="author">Student's full name</h1>
```

Then, within the <head> section of your HTML file, include a <style> tag to add CSS styling:

```
<!DOCTYPE html>
  <html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Website Template</title>
    <meta name="viewport" content="width=device-width,
    initial-scale=1.0">
    <style>
      h1.author {
      color: olive;
      text-align: center;
      font-family: Arial, sans-serif;
      }
 </style>
</head>
```

3.3. CSS Selectors: Targeting Elements and Understanding Priorities

CSS selectors are patterns used to target HTML elements and apply styles to them. They are fundamental to styling because they define **which elements** will be affected. Along with understanding selectors, it's important to know **selector specificity**, which determines which styles take priority when multiple rules apply to the same element.

3.3.1. Type Selector (Element Selector)

1. Element Selector

Targets elements by their HTML tag name.

```
p {
    color: blue;
}
```

All <p> elements will appear blue.

2. Class Selector

Targets elements with a specific class. Use a period (.) before the class name.

```
.highlight {
    background-color: yellow;
}
```

Any element with `class="highlight"` will have a yellow background.

3. ID Selector

Targets a single element with a unique ID. Use a hash (#) before the ID. ID selectors are **more specific** than type or class selectors.

```
#main-title {
    font-size: 32px;
    color: green;
}
```

Only the element with `id="main-title"` will be affected.

4. Universal Selector

Targets **all elements**.

```
* {
    margin: 0;
    padding: 0;
}
```

5. Group Selector

Targets multiple elements at once by separating them with commas.

```
h1, h2, h3 {
    font-family: Arial, sans-serif;
}
```

6. Descendant Selector

Targets elements nested inside other elements.

```
div p {
    color: green;
}
```

All <p> inside will be green.

7. Attribute Selector

Targets elements based on attributes or attribute values.

```
input[type="text"] {
    border: 1px solid #ccc;
}
```

8. Pseudo-classes and Pseudo-elements

- Pseudo-classes target elements in a specific state:

```
a:hover { color: red; }
p:first-child { font-weight: bold; }
```

- Pseudo-elements target parts of an element, like the first letter or before/after content:

```
p::first-letter {
    font-size: 24px;
    color: blue;
}
```

Selector Priorities (Specificity)

When multiple CSS rules apply to the same element, **specificity** determines which style wins:

Selector Type	Specificity Weight
Inline styles	1000
ID selector (#id)	100
Class, attribute, pseudo-class (.class, [attr], :hover)	10
Type selector (div, p)	1
Universal selector (*)	0

Table 1 – CSS selector type with its respective specificity weight

Example 1:

```
p { color: blue; }          /* specificity 1 */
.highlight { color: red; }  /* specificity 10 */
#main-title { color: green; } /* specificity 100 */
```

- If a `<p id="main-title" class="highlight">` exists, it will be **green**, because the ID selector has the highest specificity.

Understanding selectors and their priorities allows you to write CSS that is precise, avoids conflicts, and makes your web pages easier to maintain.

3.4. Text in CSS

The formatting of your text can have a significant effect on how readable your pages are.

3.4.1. Text Properties

The properties that allow you to control the appearance of text can be split into two groups:

- Those that directly affect the font and its appearance (including the typeface, whether it is regular, bold or italic, and the size of the text)
- Those that would have the same effect on text no matter what font you were using (including the color of text and the spacing between words and letters)

When choosing a typeface, it is important to understand that a browser will usually only display it if it's installed on that user's computer.

3.4.2. Typeface Terminology

When designing web pages, understanding typefaces and how they are handled in HTML is essential for creating visually appealing and readable content. A **typeface** refers to a set of characters (letters, numbers, symbols) that share a common design style. In web development, typefaces are applied through **fonts**, which control how text appears on the page.

Here are some key terms and concepts related to typefaces in HTML:

1. **Font Family**
   The font family defines the overall style of the text. Common font families include:
   - **Serif** – fonts with small lines or strokes at the ends of letters (e.g., Times New Roman, Georgia).
   - **Sans-serif** – clean fonts without extra strokes (e.g., Arial, Helvetica).
   - **Monospace** – fonts where each character takes up the same amount of horizontal space (e.g., Courier New).
   - **Cursive** – decorative fonts that resemble handwriting.
   - **Fantasy** – creative or ornamental fonts for stylistic effect.

In HTML and CSS, you can set a font family like this:

```
p {
    font-family: Arial, sans-serif;
}
```

2. **Font Weight**
   Font weight determines the thickness of the characters. Common values include:
   - normal (default)
   - bold
   - lighter
   - Numeric values from 100 (thin) to 900 (extra-bold)

Example in CSS:

```
h1 {
    font-weight: bold;
}
```

3. **Font Style**
   Font style controls whether the text is normal, italicized, or oblique:
   - normal – default upright text
   - italic – slanted style
   - oblique – similar to italic, but less refined

Example:

```
em {
    font-style: italic;
}
```

Font Size

Font size adjusts how large or small the text appears. The default size of text in a browser is 16 pixels. You can specify size using pixels (px), em units (em), percentages (%), or rem units (rem)

- pixels, px: Setting font size in pixels is the best way to ensure that the type appears at the size you intended (because percentages and ems are more likely to vary if a user has changed the default size of text in their browser.
    - `<body>` → `16px`
    - `<p>` → `16px`
    - `<h1>` → `32px`
    - `<h2>` → 24px

- percentages, %: The default size of text in a web browser is 16 pixels. Using percentages of this amount, you can create a scale where the default text size is 12 pixels, and headings are sized in relation to this.
    - `<body>` → `100%`
    - `<p>` → `100%`
    - `<h1>` → `200%`
    - `<h2>` → `150%`

- Rem and em units, ems or rem: ems or rem allows you to change the size of text relative to the size of the text in the parent element. Since the default size of text in web browsers is 16 pixels, you can use similar rules to those shown for percentages.
    - `<body>` → `1rem`
    - `<p>` → `1rem`
    - `<h1>` → `2rem`
    - `<h2>` → `1.5rem`

Example:

```
p {
    font-size: 1.3rem;
}
```

4. **Web-Safe Fonts vs. Web Fonts**
    - **Web-safe fonts** are widely supported across browsers and operating systems.
    - **Web fonts** are custom fonts loaded from services like Google Fonts or Adobe Fonts.
5. **Fallback Fonts**
   When specifying a font family, it's good practice to include fallback fonts in case the preferred font isn't available:

```
body {
    font-family: "Roboto", Arial, sans-serif;
}
```

   By understanding these typeface terms and how to use them in HTML and CSS, developers can create web pages that are not only visually consistent but also readable and accessible across different devices and browsers.

6. **Text Alignment (horizontal)**
   The **text-align** property allows you to control the alignment of text. The property can take one of four values:
   - **left**: This indicates that the text should be left-aligned.
   - **right**: This indicates that the text should be right-aligned.
   - **center**: This allows you to center text.
   - **justify**: This indicates that every line in a paragraph, except the last line, should be set to take up the full width of the containing box.

7. **Text shadow**
   The **text-shadow** property is used to create a drop shadow, which is a dark version of the word just behind it and slightly offset. It can also be used to create an embossed effect by adding a shadow that is slightly lighter than the text.

   text-shadow can use three lengths and a color for the drop shadow.

   The first length indicates how far to the left or right the shadow should fall.

   The second value indicates the distance to the top or bottom that the shadow should fall.

The third value is optional and specifies the amount of blur that should be applied to the drop shadow.

The fourth value is the color of the drop shadow.

```
text-shadow: 5px 10px 20px black;
```

Exercise 3) Exploring CSS Typefaces with AI

In this exercise, we are going to learn how to apply and customize CSS typefaces while experimenting with text alignment, shadows, weights, and sizes, using AI as a guide.

Instructions:

1. **HTML structure**
    - In index.html, add the following content in the <body>:

```
<h1 class="title">Exploring CSS Typefaces with AI</h1>
<p class="serif">This is a serif font example.</p>
<p class="sans-serif">Sans-serif font example.</p>
<p class="monospace">Monospace font example.</p>
```

2. **Generate CSS with AI**
    - Open an AI platform and give the following prompt:

      "Having the following HTML elements <copy and paste the HTML structure above>, suggest CSS for .serif, .sans-serif, and .monospace paragraphs that includes font-family, font-size, font-weight, text-align, and text-shadow to make each typeface visually distinct."

    - Copy the generated CSS into style.css.
3. **Apply and test**
    - Open index.html in a web browser to see the styled paragraphs.
    - Observe how AI's suggestions affect readability, style, and alignment.
4. **Experiment**
    - Modify the font-size, font-weight, text-shadow, or text-align to see how each property changes the text appearance.

- Try combining your own font choices with AI recommendations to create a personalized style.

5. **Reflect**
    - Write a few sentences on which typefaces you found most readable and visually appealing.
    - Describe how text alignment, shadows, weight, and size influence the overall presentation of the text.

3.5. Element appearance and spacing

Every box has three available properties that can be adjusted to control its appearance:

3.5.1. border

Every box has a border (even if it is not visible or is specified to be 0 pixels wide). The border separates the edge of one box from another.

Border Style

You can control the style of a border using the border-style property. This property can take the following values:

- **solid** a single solid line.
- **dotted** a series of square dots (if your border is 2px wide, then the dots are 2px squared with a 2px gap between each dot)
- **dashed** a series of short lines
- double two solid lines (the value of the border-width property creates the sum of the two lines)
- **groove** appears to be carved into the page
- **ridge** appears to stick out from the page
- **inset** appears embedded into the page
- **outset** looks like it is coming out of the screen
- **hidden / none** no border is shown

border values can be written in one line: border style, border size, and border color:

```
border: dashed 5px red;
```

Example 2) Add border style, size, and color to a paragraph

HTML code

```
<p class="page_border">How to work with border</p>
```

CSS code

```
.page_border{
  border: dashed 5px red;
  font-size: 30px;
}
```

output



3.5.2. margin

Margins sit outside the edge of the border. You can set the width of a margin to create a gap between the borders of two adjacent boxes.

The margin property controls the gap between boxes. Its value is commonly given in pixels, although you may also use percentages or ems.

Specify margin

You can specify values for each side of a box using: margin-top, margin-right, margin-bottom, margin-left.

Example 3) Add right margin to a paragraph

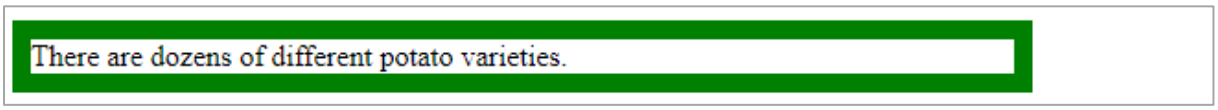
HTML code

```
<p class="m1">There are dozens of different potato
varieties.</p>
```

CSS code

```
.m1{margin-right:100px; border: solid 10px green;}
```

output



Margin with one value

A margin with one value means that the value is added to all sides (top, right, bottom, left) of the element.

Example 4) Add the same margin, 50px, to all sides of a paragraph

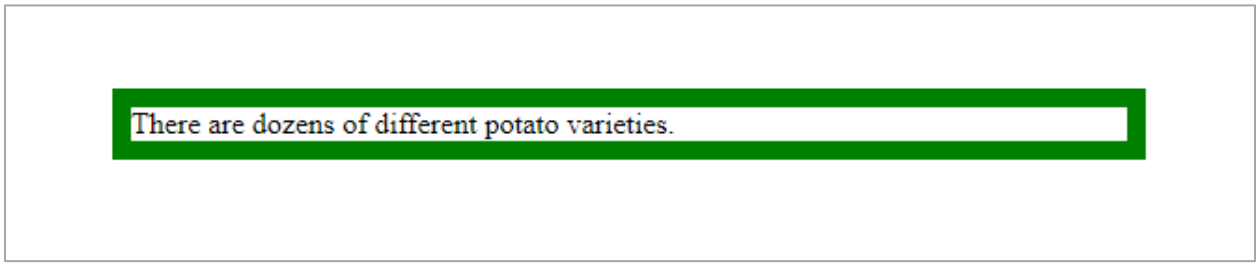
HTML code

```
<p class="m2">There are dozens of different potato
varieties. </p>
```

CSS code

```
.m2{margin: 50px; border: solid 10px green;}
```

Output



Margin with two values

A shorthand to add margin is by given two values. The first value is for the top and bottom margin, and the second value is for the left and right margin.

Example 5) Add 50px to the top and bottom of a paragraph and 100px to the right and left of the a paragraph

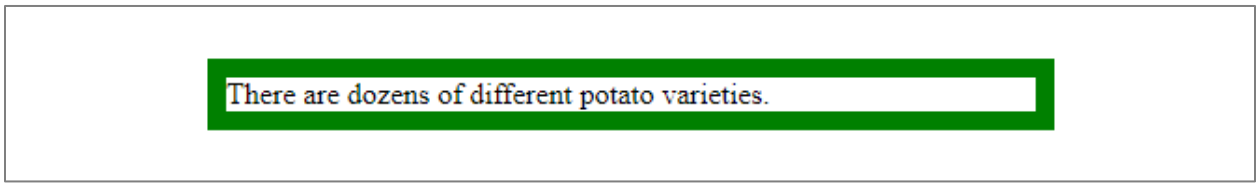
HTML code

```
<p class="m3">There are dozens of different potato
varieties. </p>
```

CSS code

```
.m3{margin: 50px 100px; border: solid 10px green;}
```

Output



Margin with four values

You can also use another shorthand with four values, where the values are in clockwise order: top, right, bottom, left

```
margin: 10px 20px 30px 5px;
```

Example 6) Create three paragraphs and apply different margin and border values to each paragraph

HTML code

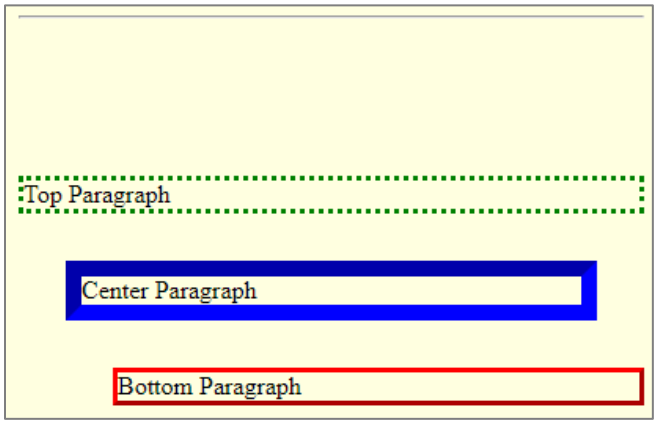
```
<p class="margin1">Top Paragraph </p>
<p class="margin2">Center Paragraph</p>
<p class="margin3">Bottom Paragraph</p>
```

CSS code

```
.margin1{
  margin-top: 100px;
  border: dotted green 3px;
}
.margin2{
  margin: 30px;
```

```
  border:inset blue 10px;
}
.margin3{
  margin-left: 60px;
  border: outset red;
}
```

Output



3.5.3. padding

Padding is the space between the border of a box and any content contained within it. Adding padding can increase the readability of its contents.

The padding property allows you to specify how much space should appear between the content of an element and its border.

The value of this property is most often specified in pixels (although it is also possible to use percentages or ems). If a percentage is used, the padding is a percentage of the browser window (or of the containing box if it is inside another box).

Specify padding

You can specify different values for each side of an element by using padding-top; padding-right; padding-bottom; padding-left.

Example 7) add 30px of left padding to a paragraph

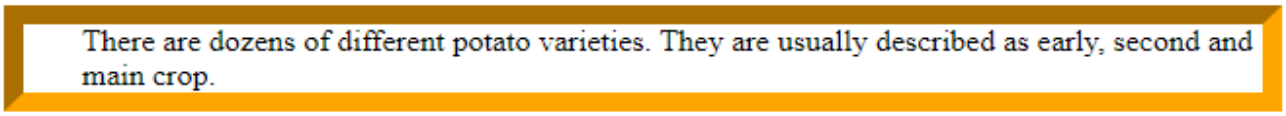
HTML code

```
<p class="padding1">There are dozens of different potato varieties. They are usually described as early, second and main crop.</p>
```

CSS code

```
.padding1{padding-left: 30px; border: 10px inset orange; }
```

Output



Padding with one value

A padding with one value means that the value is added to all sides (top, right, bottom, left) of the element.

Example 8) apply 50px of padding to all sides of an element

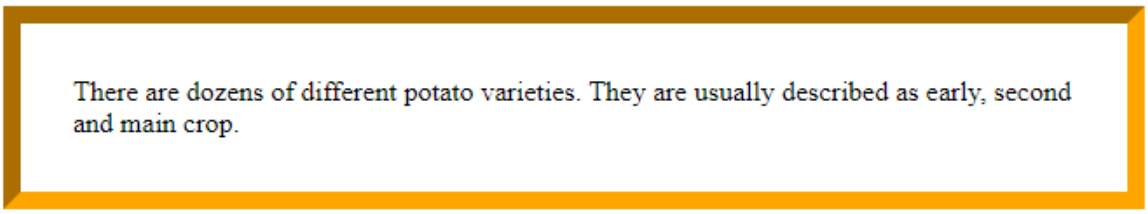
HTML code

```
<p class="padding2">There are dozens of different potato varieties. They are usually described as early, second and main crop.</p>
```

CSS code

```
.padding2{padding: 30px; border: 10px inset orange; }
```

Output



Padding with two values

A shorthand to add padding is by given two values. The first value is for the top and bottom padding, and the second value is for the left and right padding.

Example 9) apply 30px of padding to the top and bottom, and 100px to the left and right of an element

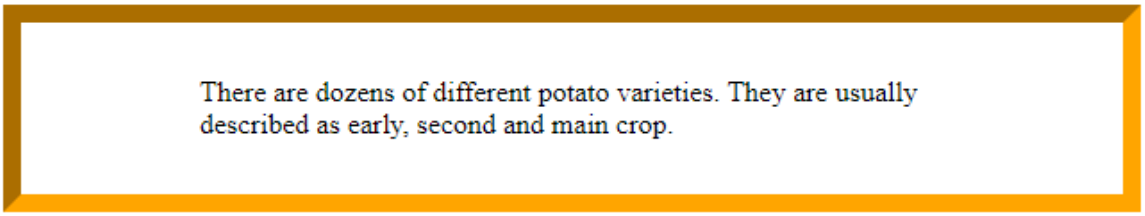
HTML code

```
<p class="padding3">There are dozens of different potato varieties. They are usually described as early, second and main crop.</p>
```

CSS code

```
.padding3{padding: 30px 100px; border: 10px inset orange; }
```

Output



Padding with four values

Another shorthand to add padding is by given four values, where the values are in clockwise order: top, right, bottom, left

Example 10) apply 10px to top padding, 5px to right padding, 30px to bottom padding, and 100px to left padding

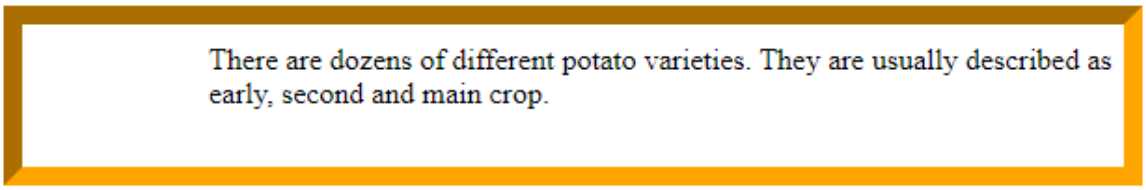
HTML code

```
<p class="padding4">There are dozens of different potato varieties. They are usually described as early, second and main crop.</p>
```

CSS code

```
.padding4{padding: 10px 5px 30px 100px ; border: 10px inset orange;}
```

Output



Example 11) Create three paragraphs and apply three different padding to each of them.

HTML code

```
<p class="one">Without Padding</p>
<p class="two">padding to 20px</p>
<p class="three">padding top by 50px</p>
```

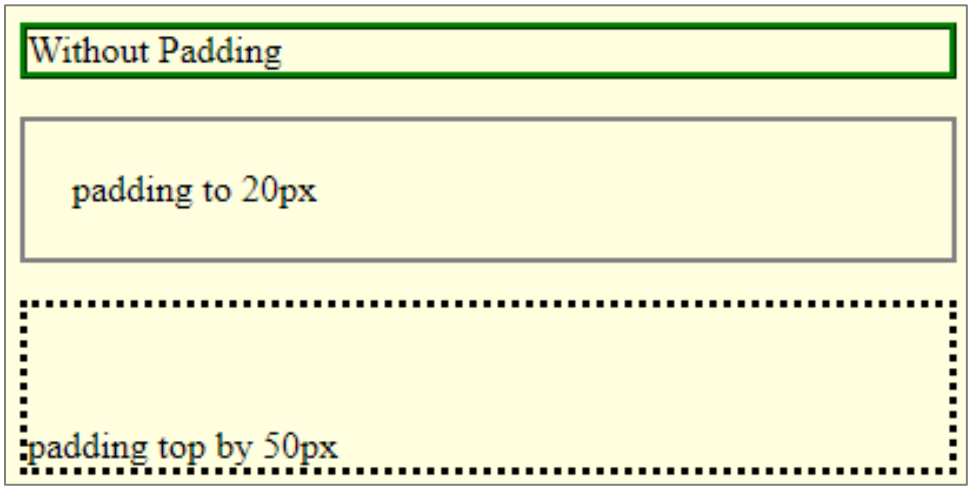
CSS code

```
.one{ border: ridge green;}

.two{
padding:20px;
border: solid gray 2px;
}
```

```
.three{
   padding-top: 50px;
   border: dotted black;
}
```

Output



3.6. CSS Layout Properties

3.6.1. Basic CSS Layout Design

In this chapter we are going to look at how to control where each element sits on a webpage and how to create attractive page layouts.

Screen size

Different visitors to your site will have different sized screens that show different amounts of information, so your design needs to be able to work on a range of different sized screens.



Dell XPS 13 Laptop

Display: 13.3 inches across screen

Display resolution: 1920 × 1080 pixels



Apple MacBook

Display: 12 inches across screen

Display resolution: 2304 × 1440 pixels

Building blocks

CSS treats each HTML element as if it is in its own box. This box will either be a block-level box or an inline box.

If one block-level element sits inside another block-level element then the outer box is known as the containing or parent element.

It is common to group a number of elements together inside a (or other block-level) element.

Static Layouts: Fixed Width

Fixed width layout designs do not change size as the user increases or decreases the size of their app window. Measurements tend to be given in pixels.

Advantages

- Pixel values are accurate at controlling size and positioning of elements.

- The designer has far greater control over the appearance and position of items on the page than with liquid layouts.
- You can control the lengths of lines of text regardless of the size of the user's window.
- The size of an image will always remain the same relative to the rest of the page.

Disadvantages

You can end up with big gaps around the edge of a page.

- If the user's screen is a much higher resolution than the designer's screen, the page can look smaller and text can be harder to read.
- If a user increases font sizes, text might not fit into the allotted spaces.
- The design works best on devices that have a site or resolution similar to that of desktop or laptop computers.
- The page will often take up more vertical space than a liquid layout with the same content.

Dynamic Layouts: Liquid Layout

The liquid layout uses percentages to specify the width of each box so that the design will stretch to fit the size of the screen.

Advantages

- Pages expand to fill the entire app window so there are no spaces around the page on a large screen.
- If the user has a small window, the page can contract to fit it without the user having to scroll to the side.
- The design is tolerant of users setting font sizes larger than the designer intended (because the page can stretch).

Disadvantages

- If you do not control the width of sections of the page then the design can look very different than you intended, with unexpected gaps around certain elements or items squashed together.
- If the user has a wide app window, lines of text can become very long, which makes them harder to read.

- If the user has a very narrow app window, words may be squashed and you can end up with few words on each line.
- If a fixed width item (such as an image) is in a box that is too small to hold it (because the user has made the window smaller) the image can overflow over the text.

Elements used to design layouts

There are multiple ways and different types of elements and properties that we can design a static or dynamic layout. We can look at the different types of elements first and then the different CSS properties that can help us to design a static and dynamic layout.

3.6.2. Display properties

The **display** property specifies the display behavior (the type of rendering box) of an element.

In HTML, the default display property value is taken from the HTML specifications or from the browser/user default style sheet. The default value in XML is inline, including SVG elements.

Note:

- The values "flex" and "inline-flex" requires the -webkit- prefix to work in Safari.
- "display: contents" does not work in Edge prior version 79.

Some of the display properties are:

Property	Description
inline	Displays an element as an inline element (like). Any height and width properties will have no effect
block	Displays an element as a block element (like <p>). It starts on a new line, and takes up the whole width

contents	Makes the container disappear, making the child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height and width values
inline-flex	Displays an element as an inline-level flex container
inline-grid	Displays an element as an inline-level grid container
inline-table	The element is displayed as an inline-level table
list-item	Let the element behave like a element

Table 2 – Display properties and its description

3.6.3. Flexbox container

The Flexible Box Layout Module makes it easier to design flexible responsive layout structure without using float or positioning. Flexbox is a one-dimension layout method for layout out items in rows or columns. It allows us to distribute space dynamically across elements of an unknown size, hence the term *flex*.

Example 12) Create a flex container for three square division:

HTML code

```
<section class="flex_container">
  <div class="sqr"></div>
  <div class="sqr"></div>
  <div class="sqr"></div>
</section>
```

CSS code

```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
}
```

```
.sqr{width: 100px; height: 100px;}
.sqr:nth-child(1){background-color: magenta;}
.sqr:nth-child(2){background-color: olive;}
.sqr:nth-child(3){background-color: orange;}
```

Output

without applying a *display: flex* to the flex container, the output will look as:



Now, if we add the *display:flex;* property to the flex container, the three division will wrap, from left to right, around the flex container

```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
}
```



We can also change the direction from right to left by adding the property *flex-direction: row-reverse;*

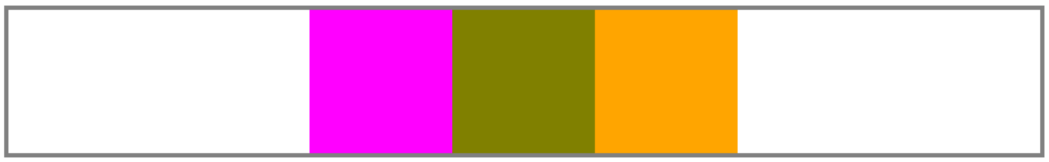
```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
   flex-direction: row-reverse;
}
```



justify-content property

The CSS justify-content property defines how the browser distributes space between and around content items along the main-axis of a flex container, and the inline axis of a grid container. **justify-content** uses different values such as: **start, center, space-between, space-around,** and **space-evenly**

```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
   flex-direction: row;
   justify-content: center;
}
```



```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
   flex-direction: row;
   justify-content: space-between;
}
```



```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
   flex-direction: row;
   justify-content: space-around;
}
```



```
.flex_container{
   border: solid gray;
   width: 80%;
   margin: 10%;
   display: flex;
   flex-direction: row;
   justify-content: space-evenly;
}
```



Flex-wrap property
The flex-wrap property specifies whether the flexible items should wrap or not.

Example 13) from the previous example, let change the **width** of the divisions to **500px**.



For this case, if we change the browser window to a smaller view, all three divisions it will squeeze to fit the width of the *flex_container*



If we do not want the divisions to squeeze, but instead, we want to keep the division's width and have the divisions to wrap around the *flex_container*, then we can add the **flex-wrap:wrap;** property to the *flex_container*



Example 14) from the previous example, if we set the **height** of the `flex_container` to **200px** and the **flex-direction** to **column**,

```
.flex_container{
      border: solid gray;
      width: 80%;
      margin: 10%;
      height: 200px;
      display: flex;
      flex-direction: column;
      justify-content: space-evenly;
    }
```

all three division at the container will squeeze to fit the height of the container. The output will look as:



Now, if we want to keep the height of each divisions, we can use **flex-wrap:wrap** to allow the flexible divisions to wrap within the *flex_container*

```
.flex_container{
  border: solid gray;
  width: 80%;
  margin: 10%;
  height: 200px;
  display: flex;
  flex-direction: column;
  justify-content: space-evenly;
  flex-wrap: wrap;
}
```



Align-item property

Align-items property sets the align-self value on all direct children as a group. In flexbox, it controls the alignment of items inside the flex container on the cross axis.

Example 15) using the three divisions, set the divisions to align to the end of the flex container.

Without using the *align-item* property,

```
.flex_container{
  border: solid gray;
  width: 80%;
  height: 200px;
  margin: 10%;
  display: flex;
  flex-direction: row;
  justify-content: center;
}
```



By adding the ***align-items:flex-end***; to *flex_container*

```
.flex_container{
  border: solid gray;
  width: 80%;
  height: 200px;
  margin: 10%;
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items:flex-end;
}
```



We can also set the divisions to the center of the cross axis of the flex container by using **align-items:center;**

```
.flex_container{
  border: solid gray;
  width: 80%;
  height: 200px;
  margin: 10%;
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items:center;
}
```



align-content property

align-content property sets the distribution of space between and around content items along a flexbox's cross-axis or a grid's block axis. The align-content property modifies the behavior of the flex-wrap property. It is similar to align-items, but instead of aligning flex items, it aligns flex lines. It sets the distribution of space between and around content items along a flexbox's cross-axis or a grid's block axis.

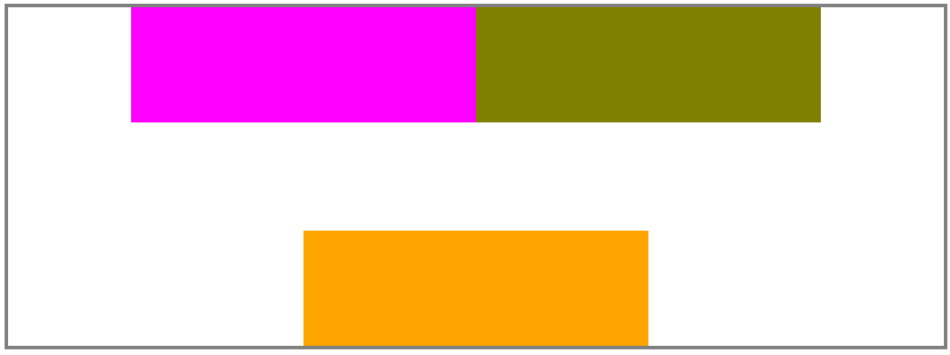
Note: There must be multiple lines of items for this property to have any effect!

Example 16) use the previous example and change the **width** of the division to **300px**

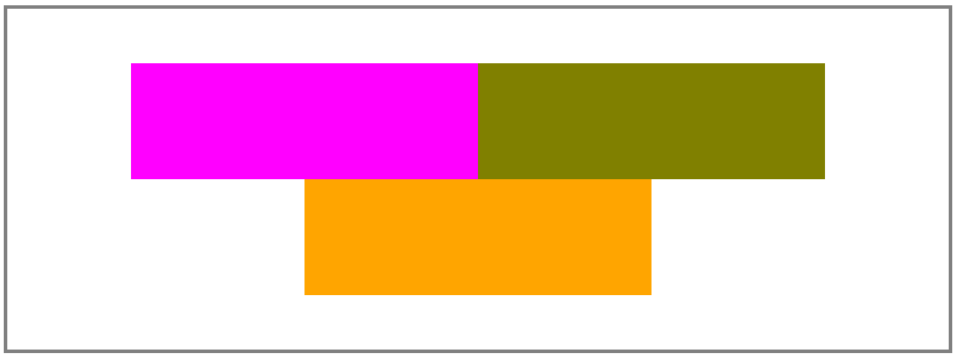
```
.flex_container{
      border: solid gray;
      width: 80%;
      height: 300px;
      margin: 0 auto;
      display: flex;
      flex-direction: row;
      justify-content: center;
      align-items:flex-end;
      flex-wrap: wrap;
    }
.sqr{width: 300px; height: 100px;}
```



Now, by adding the property ***align-content: space-between;***



Now, if we change ***align-content: center;***

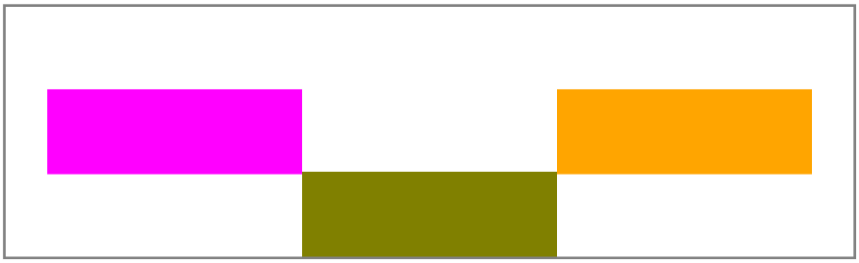


align-self property

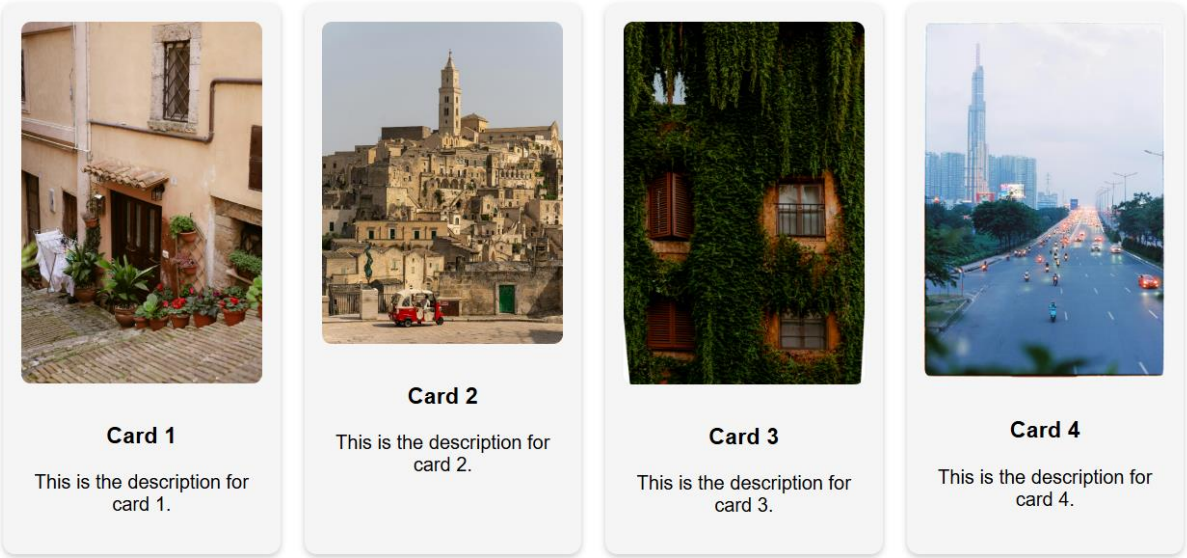
The *align-self* CSS property overrides a grid or flex item's align-items value. In Grid, it aligns the item inside the grid area. In Flexbox, it aligns the item on the cross axis. The *align-self* property specifies the alignment for the selected item inside the flexible container.

Example 17) using the previous example, align the second division to the bottom of the flex container

```
.flex_container{
        border: solid gray;
        width: 80%;
        height: 300px;
        margin: 0 auto;
        display: flex;
        flex-direction: row;
        justify-content: center;
        align-items:center;
        flex-wrap: wrap;
}
.sqr:nth-child(2){background-color: olive; align-self:
flex-end;}
```



Exercise 4: In the AI prompt, create a prompt that generates HTML and CSS code for a row of 4 cards, each with an image, a title, and a description.



For this activity, select and download four images with the same orientation and save them in a folder named images inside your project directory.

```
Intro_CSS/
 |
 ├──── index.html
 └──── style.css
 └──── images /
       └──── image1.jpg
       └──── image2.jpg
       └──── image3.jpg
       └──── image4.jpg
```

Copy and paste the HTML and CSS code generated by AI in their respective files.

Open the web browser and observe the layout generated by AI.

3.7. Navigation Bars

A navigation bar needs standard HTML as a base, which can be built using the <ul> and <li> elements or <nav> element. To create a navigation bar using <ul> and

3.7.1 Vertical Navigation Bars

To create a basic vertical navigation bar using **<ul>,** we can list each hyperlink using **<a>** tag as the following:

```
<ul class="navbar">
  <li><a href="#home">Home</a></li>
  <li><a href="#nav">Tabs</a></li>
  <li><a href="#flex">Flex-Wrap</a></li>
  <li><a href="#social">Social Media</a></li>
</ul>
```

To take off the bullet of each list, we can set the **list-style-type** to **none** of the <ul>:

```
.navbar{
  list-style-type: none;
}
```

To make the vertical navigation bar with a purple background-color, we can write CSS codes to make changes to the hyperlinks in the list item. Also, **display: block;** property sets the <a> elements in a block, column, format because <a> element by default is an inline element

```
.navbar li a {
  display: block;
  padding: 8px 16px;
  color: white;
  background-color: purple;
  width: 50%;
  text-decoration: none;
}
```

After it, we can change the background color of the links when the user clicks on them:

```
.navbar li a:hover {
  background-color: gray;
  color: white;
}
```

The output will look like the following:



3.7.2. Horizontal Navigation Bars

To create a horizontal navigation bar, we can use inline element such as **<nav>** element and have <a> elements within it.

```
<nav class="nav_div">
  <a href="#home">Fantasy Shoes</a>
  <a href="#products">Season's Shoes</a>
  <a href="#sale">Winter Sale</a>
  <a href="#social">Follow Us</a>
</nav>
```



Once we have the HTML code set, in CSS, we can set the hyperlink container with background-color, the width, the height, and the font-size:

```
.nav_div{
    background-color: darkgray;
    width: 100%;
    font-size: 1.5em;
    }
```

After it, we can set hyperlink with left border, height, width, font color, center the text, and add the padding. We also set the hyperlink to float from the left to right without underline:

```
.nav_div a {
  display:inline-block;
  color: white;
  background-color: darkgray;
  text-align: center;
  padding: 10px 20px;
  text-decoration: none
}
```

After it, we can change the background color of the links when the user hover on them:

```
.nav_div a:hover {
  background-color: orange;
}
```

The output will look like the following:



3.7. Color in CSS

3.7.1. Colors

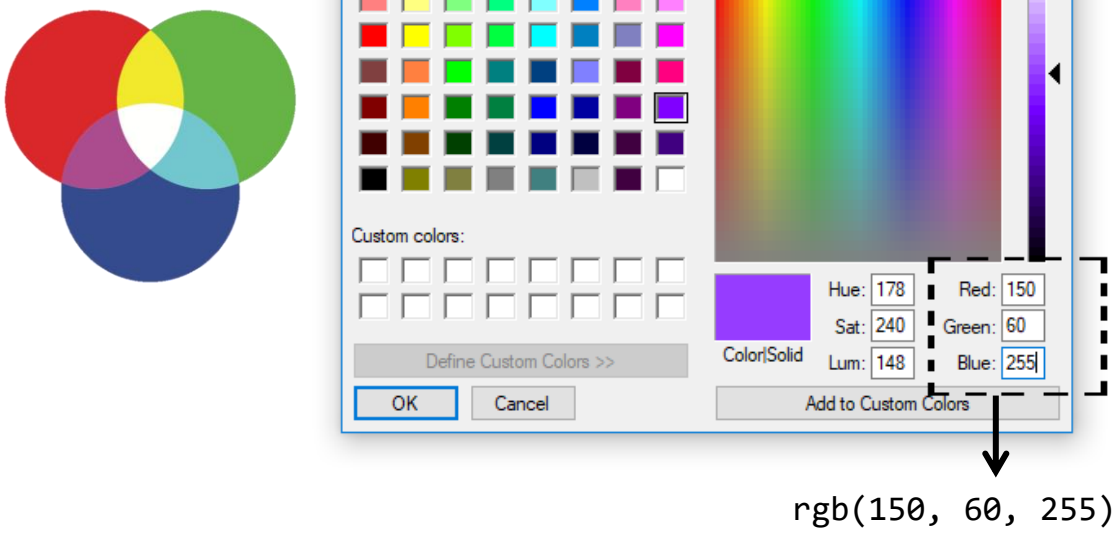
The color property allows us to specify the color of text inside an element. We can specify any color in CSS in one of three ways:

Color Names

There are 147 predefined color names that are recognized by browsers. For example: darkcyan

RGB Values

These express colors in terms of how much red, green and blue are used to make it up. Every color on a computer screen is created by mixing amounts of red, green, and blue. To find the color you want, we can use a color picker.



Example 17) find each RGB color the Microsoft logo:



To find a specific color, we can use the color picker tool available in most graphic applications. One simple graphic app that comes with Windows is Paint. Using Paint, we can use the color picker tool to identify the four colors of the Microsoft logo. Alternatively, you can open an AI prompt, upload an image, and ask the AI to determine the RGB values of the logo.

- Red: rgb(242,80,34)
- Green: rgb(127,186,0)
- Blue: rgb(1,164, 239)
- Yellow: rgb(255, 185,1)



The RGB values in CSS to replicate the four colors are as follows:

HTML code

```
<section class="rgb">

   <div class="red"></div>
   <div class="green"></div>
   <div class="blue"></div>
   <div class="orange"></div>
</section>
```

CSS code

```
.rgb{margin: 2em; display: inline-block;}
.red,.green,.blue,.orange{
width:100px;
height:100px;
margin: 20px;
float: left;
}
.red{background-color:rgb(242,80,34)}
.green{background-color:rgb(127,186,0)}
.blue{background-color:rgb(1,164,239)}
.orange{background-color:rgb(255,185,1)}
```

rgba( ) color

The rgba( ) function define colors using the red-green-blue-alpha (RGBA) model where alpha specifies the opacity of the color. Alpha color goes from 0 to 1, 0 means fully transparent (0% of opacity) and 1 means fully opaque (100% of opacity). The syntax of the code is:

rgba(*red, green, blue, alpha*)

For example, for color **rgb(150, 60, 255)**with 35% of opacity, we can write the code as:

rgba(150, 60, 255,0.35)

HEX Codes

HEX colors are one of the most common ways to define colors in CSS. A HEX color code starts with a hash symbol (#) followed by six hexadecimal characters (numbers 0–9 and letters A–F). These characters represent the intensity of red, green, and blue (RGB) in a color.

1. Understanding HEX Structure

A six-digit HEX color code is written like this:

#RRGGBB

- **RR** → Red value
- **GG** → Green value
- **BB** → Blue value

Each pair ranges from 00 (no color) to FF (maximum intensity).

For example:

- #FF0000 → Red
- #00FF00 → Green
- #0000FF → Blue
- #000000 → Black
- #FFFFFF → White

2. Using HEX Colors in CSS

You can apply HEX colors to many CSS properties such as color, background-color, border, and more.

Example:

```
p {color: #333333;}

body { background-color: #F4F4F4; }

button {
  background-color: #007BFF;
  border: 1px solid #0056B3;
}
```

3. Three-Digit Shorthand

Some HEX colors can be shortened from six digits to three digits if each pair has identical characters.

Example:

- #FFFFFF → #FFF
- #000000 → #000
- #FFCC00 → #FC0

Example in CSS:

```
h1 {   color: #F00;  /* Red */ }
```

4. HEX with Transparency (8-Digit HEX)

CSS also supports 8-digit HEX values where the last two digits represent opacity (alpha channel).

Format:

```
#RRGGBBAA
```

Example:

```
div {
  background-color: #0000FF80;
  /* Semi-transparent blue*/
}
```

Here, 80 controls the transparency.

HEX colors provide a precise and widely supported way to define colors in CSS. By understanding how the red, green, and blue values work together, you can easily create and customize colors for your web designs.

3.8. Image manipulation in CSS

3.8.1. Position Property

The position CSS property sets how an element is positioned in a document. For position: relative, absolute, or sticky, the top, right, bottom, and left properties determines the final location of positioned elements.

position: static;

HTML elements are positioned static by default.
Static positioned elements are not affected by the top, bottom, left, and right properties.

An element with position: static; is not positioned in any special way; it is always positioned according to the normal flow of the page.

position: relative;

An element with position: relative; is positioned relative to its normal position.

Setting the top, right, bottom, and left properties of a relatively-positioned element will cause it to be adjusted away from its normal position. Other content will not be adjusted to fit into any gap left by the element.

position: absolute;

An element with position: absolute; is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like fixed).

However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.

Note: A "positioned" element is one whose position is anything except static.

Exercise) Create container with a back image with position:relative; and a message with position:absolute;

We can create a container for the back image using ** ** element

```
<div class="image">
      <img src="image.jpg">
</div>
```

CCS code

```
img{ width: 100%; height: 100%;}
.image{
  width: 80%;
  height: 450px;
  margin: auto;
  position: relative;
}
```

After it, we can add the text message using another or <p> and use position:absolute;

HTML code

```
<div class="message"> Creating Smartphone Apps</div>
```

CSS code

```
.message{
  position: absolute;
  top: 10%;
  left: 20%;
  width: 60%;
  background-color: rgba(0, 0, 0, 0.3);
  color: white;
  text-align: center;
  padding: 10px;
  font-size: 20px;
}
```

position: sticky;

An element with position: sticky; is positioned based on the user's scroll position.

A sticky element toggles between relative and fixed, depending on the scroll position. It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed).

Exercise 5) Mastering the CSS position Property Using AI

Objective

Practice using the following CSS position values:

- static
- relative
- absolute
- fixed
- sticky

You will use **AI as your coding assistant** to help generate variations, debug, and explain behavior.

Part 1: Starter Code

Create an HTML file called **position-practice.html** and paste this:

```
<!DOCTYPE html>
<html>
<head>
      <title>CSS Position Practice</title>
</head>
<body>
    <h1>CSS Position Practice</h1>
    <div class="container">
        <div class="box box1">Box 1</div>
        <div class="box box2">Box 2</div>
        <div class="box box3">Box 3</div>
        <div class="box box4">Box 4</div>
    </div>
```

```
</body>
</html>
```

Create a CSS file called **position-style.css** and paste this:

```
body {
      font-family: Arial, sans-serif;
      height: 2000px;
}
.container {
       position: relative;
       width: 500px;
       height: 300px;
       border: 3px solid black;
       margin: 50px auto;
}
.box {
       width: 100px;
       height: 100px;
       color: white;
       text-align: center;
       line-height: 100px;
       font-weight: bold;
}
.box1 {background-color: red;}
.box2 {background-color: blue;}
.box3 {background-color: green;}
.box4 {background-color: purple;}
```

Part 2: Tasks (Use AI as Your Assistant)

- **Task 1 – Relative Position**

1. Set **Box 1** to position: relative.
2. Move it:
    - 30px from the top
    - 40px from the left

Ask AI: *"Why does relative positioning keep the original space?"*

Record AI answers and write your understanding.

Task 2 – Absolute Position

1. Set **Box 2** to position: absolute.
2. Place it:
    - 0px from the top
    - 0px from the right

Ask AI: *"Why is my absolute element positioned relative to the container instead of the page?"*

Record AI answers and write your understanding.

Task 3 – Fixed Position

1. Set **Box 3** to position: fixed.
2. Place it:
    - 20px from bottom
    - 20px from right

Scroll the page.

Ask AI: *"Why does fixed stay visible while scrolling?"*

Record AI answers and write your understanding.

Task 4 – Sticky Position

1. Add this above the container:

```
<h2 class="sticky-header">Sticky Header</h2>
```

2. Add CSS:

```
.sticky-header {
    position: sticky;
```

```
        top: 0;
        background-color: orange;
        padding: 10px;
    }
```

Scroll the page.

Ask AI: *"What is the difference between sticky and fixed positioning?"*

Record AI answers and write your understanding.

Part 3: AI Challenge Prompts

Use AI to:

1. Generate a real-world example using absolute positioning.
2. Debug overlapping elements.
3. Explain the stacking order using z-index.
4. Create a mini layout combining relative and absolute.

Example AI Prompt:

"Create a navigation bar using fixed positioning and a badge notification using absolute positioning."

Insert the generated code into your position-practice.html file and clearly label each exercise as 1, 2, 3, and 4 using heading tags such as <h1> or <h2>.

Bonus Challenge

Ask AI to: *"Modify this layout to create a floating chat button in the bottom-right corner of the screen."*

Reflection Questions

1. When should you use relative instead of absolute?
2. What happens if no parent has position: relative?
3. Why can fixed sometimes cause layout overlap issues?
4. When is sticky most useful?

Fixed navigation bar

Fixed navigation bar happens when the navigation bar remain at the top or bottom of the app, even when the user moves the page. For this, you can use the CSS property `position: sticky;` to the element that contains the hyperlinks:

```
position: fixed;
top: 0;
```

position:fixed; and **top: 0;** will make the element to have a position fixed to the top of the app.

Example 18) Create a navigation bar that will stick on the top of the app.

We can create the hyperlink using ** **

```
<nav class="nav_top">
  <a href="#home">Home</a>
  <a href="#nav">Tabs</a>
  <a href="#flex">Flex-Wrap</a>
  <a href="#social">Social Media</a>
</nav>
```

Once we have the HTML code set, in CSS, we can set the hyperlink container with background-color, the width, the height, the font-size, and the position:

```
.nav_top{
    background-color: orange;
    font-size: 1.2em;
    height: 3em;
    position: sticky;
    top: 0;
}
```

After it, we can set hyperlink with left border, height, font color, center the text, and add the padding. We also set the hyperlink to float from the left to right without underline:

```
.nav_top a {
    float: left;
    color: white;
    text-align: center;
    text-decoration: none;
    padding: 1em 2em;
}
```

After it, we can change the background color of the links when the user clicks on them:

```
.nav_top a:hover{
    background-color: olive;
    font-weight: bold;
}
```

If you need the navigation bar stick in the bottom, you can change **top: 0;** to **bottom: 0;**

3.8.2. Background properties

Background images and linear gradient

CSS gradient display smooth transition between two or more specified colors. Those colors are the colors that we want to render smooth transition among the grading space. For example, we can create a **linear-gradient** of the red and blue color from top to bottom as the following:

```
<div class="gradient1"></div>
```

CSS code

```
.grading1 {
  height: 200px;
  background-image: linear-gradient(red, blue);
}
```



linear-gradient can set the color from left to right. For this, we can add **to right** value to the background-image property as the following:

background-image: linear-gradient(to right, red, blue);



We can also set the linear-gradient from bottom-left to top-right:

background-image: linear-gradient(to top right, red, blue);



Hero image

A hero image is a large image with text places on the top of it. For this, we can use the position absolute and relative.

Example 19) Create an advertisement that includes an image with text overlaid on top of it. Use a hero image as the background for the design.



First, we create a container with a background image:

```
<div class="heroImage">

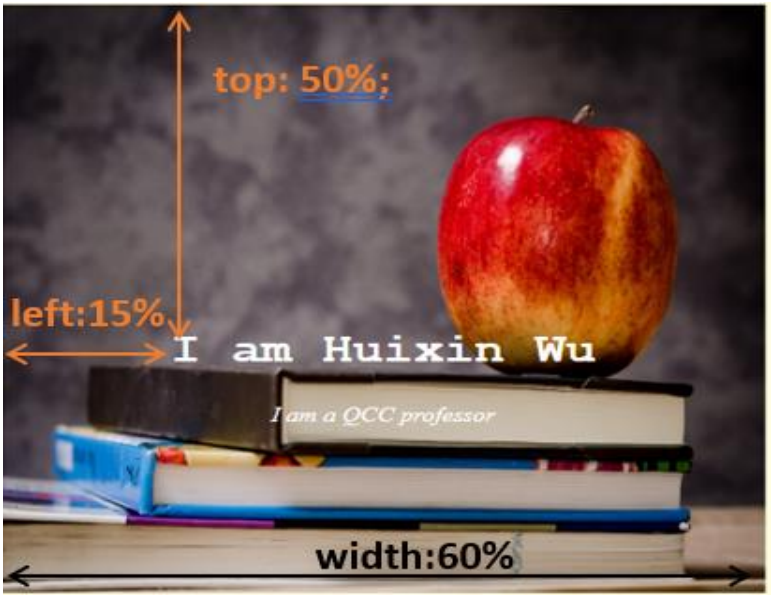
</div>
```

We can insert a background image from a CSS using **background-image** property. Also, we can align the background image using **background-position: center;** and adjust the width and height using **background-size: 60% 100%;** which 60% is the width and 100% is the height.

CSS code

```
.heroImage{
  background-image: url("apple.jpg");
  background-repeat: no-repeat;
  background-position: center;
  background-size: 60% 100%;
  height: 450px;
  position: relative;
}
```

Once we have the container and the background image set, we can create the text that will sit on top of the background image. We can name the text container **heroText**



HTML code

```
<div class="heroImage">
   <div class="heroText">
        <h1>I am Huixin Wu</h1>
        <i>I am a QCC professor</i>
   </div>
</div>
```

CSS code

```
.heroText{
  position: absolute;
  top: 50%;
  left: 15%;
  width: 70%;
  color: white;
  text-align: center;
}
```

A text on top of an image makes the text information very hard to write, for this, we can add linear-gradient to the background image. We can set a red to black gradient with 60% of opacity from left to right:



CSS code

```
.heroImage{
  background-image: linear-gradient(to right,
  rgba(255,0,0,0.6),rgba(0,0,0,0.6)), url("apple.jpg");
  background-position: center;
  background-repeat: no-repeat;
  background-size: 100% 100%;
  height: 450px;
  position: relative;
}
```

After it, we can add a Hire Me! link to the text.



HTML code

```
<h3>Hero Image </h3>
  <div class="heroImage">
    <div class="heroText">
      <h1>I am Huixin Wu</h1>
      <i>I am a QCC professor</i>
      <a href="#" class="submit">Hire Me!</a>
    </div>
  </div>
```

CSS code

```
.submit{
  display:block;
  padding: 10px 25px;
  color: black;
  background-color: rgba(255,255,255,0.6);
  font-weight: 600;
  font-size: 20px;
  margin-top: 20px;
```

```
  text-align: center;
  border-radius: 10px;
  text-decoration: none;}
  }
```

3.8.3. z-index property

The z-index property specifies the stack order of an element.

An element with greater stack order is always in front of an element with a lower stack order.

Note: z-index only works on positioned elements (position: absolute, position: relative, position: fixed, or position: sticky) and flex items (elements that are direct children of display:flex elements).

Note: If two positioned elements overlap without a z-index specified, the element positioned last in the HTML code will be shown on top.

Example 20) Use z-index on two overlap images and one text

To do so, we first use a <figure> element to hold two images and one paragraph

```
<figure>
   <img src="images/nyc4.jpg" alt="New York City"
   class="nyc1">
   <img src="images/nyc1.jpg" alt="New York City"
   class="nyc2">
   <p class="nyc_txt">New York City</p>
</figure>
```

Once we have the HTML layout, in the CSS file, we can use z-index to set the order of how the images and paragraph display on the <figure> element. For example, we can set the first image, with class="nyc1", to the back of the other image in the paragraph using z-index: -1. After it, the second image can have z-index:0 and the paragraph with z-index: 1.

CSS code

```
figure{width: 500px; height: 400px; position: relative;}
.nyc1{ width:100%;position: absolute; z-index: 0;}
.nyc2{ width:50%;position: absolute; border: ridge white
1em; z-index:1; }
.nyc_txt{font-size: 4em; position:absolute; z-index:2;}
```

The output in the browser looks as:



3.8.4. Filter images

Filter property adds visual effects to an element. Some of the filter attributes are: blur, contrast, sepia, grayscale, and invert.

blur(px)

blur applies a blur effect to an element. The value, in pixels *px*, between the parenthesis defines the blurriness of the element, a larger value will create more blur.

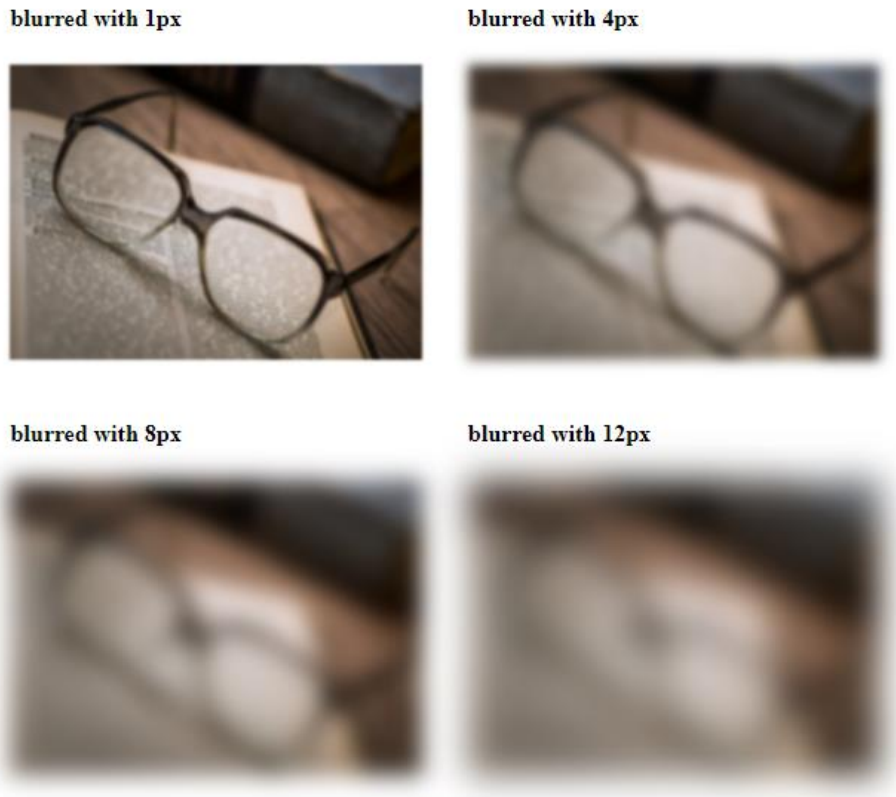
Example 21) Create four different blur effects to an image with 1px, 4px, 8px, and 12px.

HTML code

```
<div class="filterContainer">
     <div class="filter1">
      <h4>blurred with 1px</h4>
      <img src="book.jpg" style="filter:blur(1px);"/>
     </div>
     <div class="filter2">
      <h4>blurred with 4px</h4>
      <img src="book.jpg" style="filter:blur(4px);"/>
     </div>
     <div class="filter3">
      <h4>blurred with 8px</h4>
      <img src="book.jpg" style="filter:blur(8px);"/>
     </div>
     <div class="filter4">
       <h4>blurred with 12px</h4>
      <img src="book.jpg" style="filter:blur(12px);"/>
     </div>
</div>
```

CSS code

```
.filterContainer{height:700px;}
.filter1, .filter2, .filter3, .filter4{
   width: 40%; height: 40%;
   padding:2%; float:left;
}
```



contrast(%)

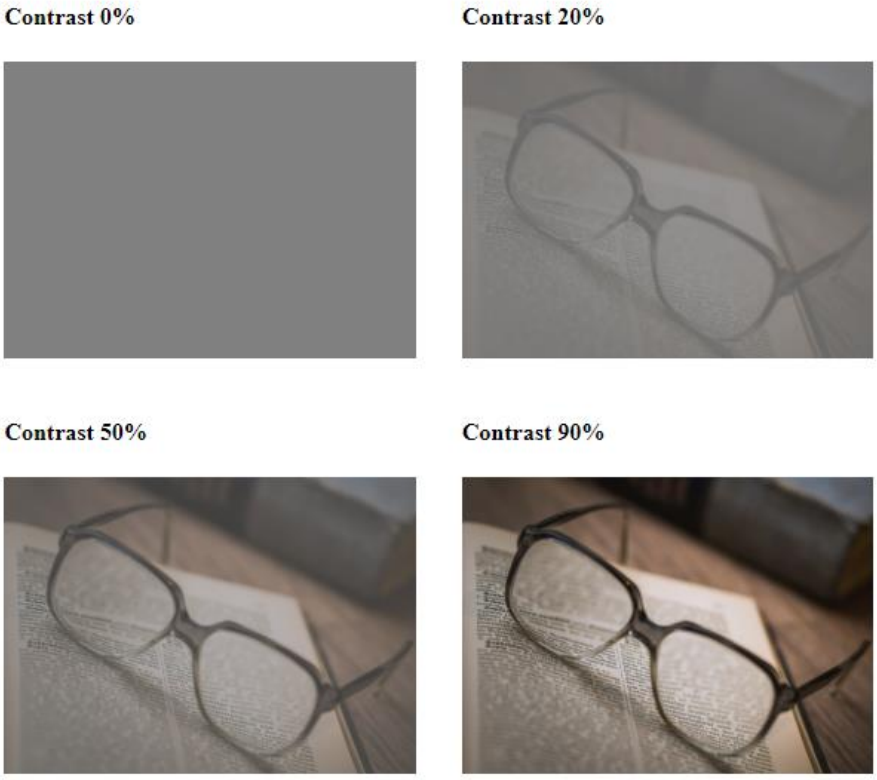
contrast adjusts the contrast of an element. If the element is used as the container for an image, then contrast will adjust the contrast of the image.

contrast(0%) will make the image completely black and **contrast(100%),** which is by default, will show the original image, and values over 100% will provide results with more contrast

HTML code

```
<div class="filterContainer">
  <div class="filter1">
      <h4>Contrast 0%</h4>
      <img src="book.jpg" tyle="filter:contrast(0%);"/>
  </div>
  <div class="filter2">
      <h4>Contrast 20%</h4>
      <img src="book.jpg"style="filter:contrast(20%);"/>
  </div>
```

```
  <div class="filter3">
      <h4>Contrast 50%</h4>
      <img src="book.jpg" style="filter:contrast(50%);"/>
  </div>
  <div class="filter4">
      <h4>Contrast 90%</h4>
      <img src="book.jpg" style="filter:contrast(90%);"/>
  </div>
</div>
```

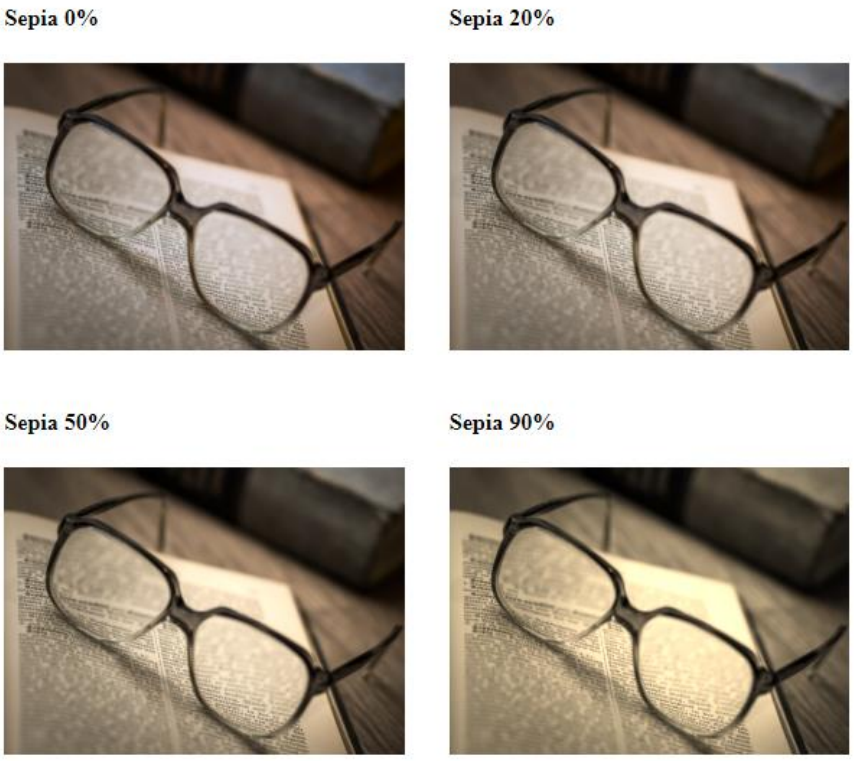


sepia(%)

sepia converts the element to sepia color. **sepia(0%)** is default means that there is no sepia applies to the element, and **sepia(100%)** will make the image completely sepia. For this attributes, negative values are not allowed.

HTML code

```
<div class="filterContainer">
  <div class="filter1">
      <h4>Sepia 0%</h4>
      <img src="book.jpg" style="filter:sepia(0%);"/>
  </div>
  <div class="filter2">
      <h4>Sepia 20%</h4>
      <img src="book.jpg" style="filter:sepia(20%);"/>
  </div>
  <div class="filter3">
      <h4>Sepia 50%</h4>
      <img src="book.jpg" style="filter:sepia(50%);"/>
  </div>
  <div class="filter4">
      <h4>Sepia 90%</h4>
      <img src="book.jpg" style="filter:sepia(90%);"/>
  </div>
</div>
```



grayscale(%)

grayscale converts the element to loose its black and white color. **grayscale(0%)** is default and it means that there isn't any changes in its grayscale. **grayscale(100%)** will make the image completely gray, black and white. This property does not allowed negative values.

HTML code

```
<div class="filterContainer">
  <div class="filter1">
      <h4>Grayscale 0%</h4>
      <img src="book.jpg"style="filter:grayscale(0%);"/>
  </div>
  <div class="filter2">
      <h4>Grayscale 20%</h4>
      <img src="book.jpg"style="filter:grayscale(20%);"/>
  </div>
  <div class="filter3">
      <h4>Grayscale 50%</h4>
      <img src="book.jpg"style="filter:grayscale(50%);"/>
  </div>
  <div class="filter4">
      <h4>Grayscale 90%</h4>
      <img src="book.jpg"style="filter:grayscale(90%);"/>
  </div>
</div>
```

Grayscale 0%



Grayscale 20%



Grayscale 50%



Grayscale 90%



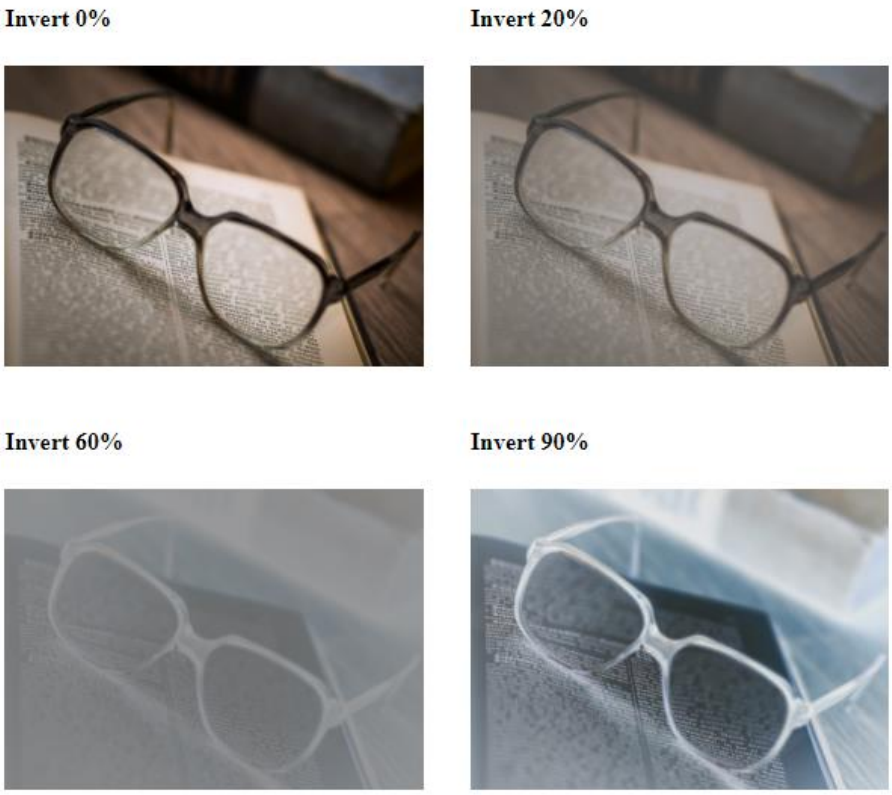
invert(%)

invert( ) inverts the sample color of the element. **invert(0%)** is default and means that there isn't any color inversion in the element. **invert(100%)** will make the image will invert completely its sample color.

HTML code

```
<div class="filterContainer">
  <div class="filter1">
      <h4>Invert 0%</h4>
      <img src="book.jpg" style="filter:invert(0%);"/>
  </div>
  <div class="filter2">
      <h4>Invert 20%</h4>
      <img src="book.jpg" style="filter:invert(20%);"/>
  </div>
  <div class="filter3">
      <h4>Invert 60%</h4>
      <img src="book.jpg" style="filter:invert(60%);"/>
  </div>
```

```
  <div class="filter4">
      <h4>Invert 90%</h4>
      <img src="book.jpg" style="filter:invert(90%);"/>
  </div>
</div>
```



3.9. Understand CSS animation

3.9.1. CSS transitions

CSS transitions are one of the ways we can create animation with CSS, even though they are not called animations. By definition, animation means causing change over time and transitions absolutely do that. They just do it in a little bit of a different way than CSS keyframe animations do.

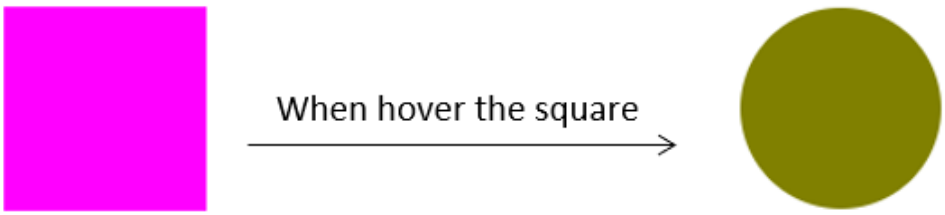
transition as property

To create a transition effect using the ***transition*** property, we must specify two things:

- The CSS property you want to add an effect to
- The duration of the effect

Note: If the duration part is not specified, the transition will have no effect, because the default value is 0.

Example 22) add transition property of 3 seconds to a square figure. The square figure changes to a circle when it is hovered



HTML code

```
<div class="square">  </div>
```

CSS code

```
.square{height:100px; width:100px; background-
color:magenta; transition: 3s;}
.square:hover{background-color: olive; border-radius:
50%; }
```

We can also transition a specific property of an element just by adding the property name to the transition. To do so, we just need to make sure that the property is already part of the element.

Example 22a) from the previous example, transition the background-color of the element

CSS code

```
.square{height:100px; width:100px; background-
color:magenta; transition: background-color 3s;}
.square:hover{background-color: olive; border-radius:
50%; }
```

We can also use transition using different properties. We separate the properties using a comma.

Example 22b) from the previous example, add a transition of 1 second to the **border-radius**

CSS code

```
.square{height:100px; width:100px; background-
color:magenta; transition: background-color 3s, border-
radius 1s;}
.square:hover{background-color: olive; border-radius:
50%; }
```

3.9.2. CSS transform property

The transform property applies a 2D or 3D transformation to an element. This property allows you to rotate, scale, move, skew, etc., elements.

Transform property has values as:

- **translate(*x,y*):** Defines a 2D translation
- **translateX(*x*):** Defines a translation, using only the value for the X-axis
- **translateY(*y*):** Defines a translation, using only the value for the Y-axis
- **scale(*x,y*):** Defines a 2D scale transformation
- **rotate(*angle*):** Defines a 2D rotation, the angle is specified in the parameter

For any CSS keyframe animation we create, we must first define the animation. Basically, tell CSS what it is that should happen and we also need to assign that animation to a specific element or elements in your HTML. We can do these two steps in any order.

Example 23) create a web app that when the mouse hovers on the app, a message zooms in from the app



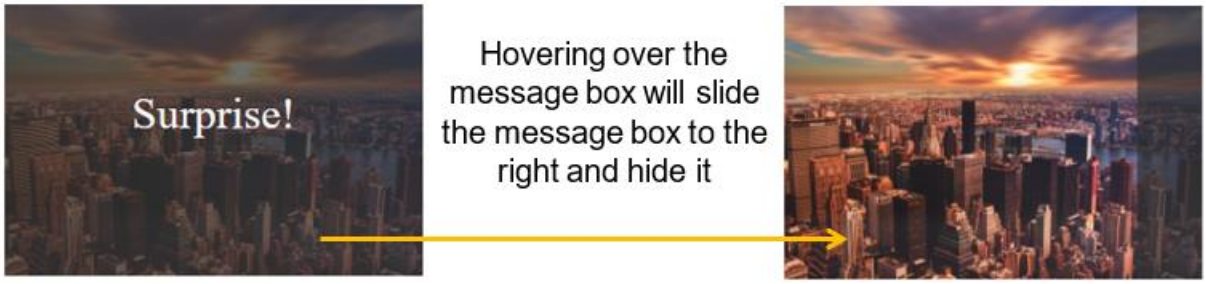
HTML code

```
<!-- zoom animation-->
<section class="zoomContainer">
   <img src="NYC.jpg" alt="photo of NYC at night">
   <div class="zoomMSG">Surprise!</div>
</section>
```

CSS code

```
.zoomContainer{ width: 300px; position: relative;}
.zoomMSG {
    position: absolute;
    top: 0%; bottom: 0%; right: 0%; left:0%;
    background-color: rgba(50,50,50,0.7);
    font-size: 2em;
    color: whitesmoke;
    text-align: center;
    padding-top: 20%;
    transition: ease 800ms;
}
.zoomMSG{
   transform: scale(0);
}
.zoomContainer:hover .zoomMSG{
    transform: scale(1);
}
```

Example 24) create a web app that when the mouse hovers on the app, a message with slides out from the app



HTML code

```
<!-- slide animation-->
<section class="slideContainer">
       <img src="NYC.jpg" alt="photo of NYC at night">
       <div class="slideMSG">Surprise!</div>
</section>
```

CSS code

```
.slideContainer{
    width: 300px;
    position: relative;
    overflow: hidden;
}
.slideMSG{
    position: absolute;
    top: 0%; bottom: 0%; right: 0%; left:0%;
    background-color: rgba(50,50,50,0.7);
    font-size: 2em;
    color: whitesmoke;
    text-align: center;
    padding-top: 20%;
    transition: ease 800ms;
}
.slideMSG:hover{
    transform: translateX(300px);
}
```

3.9.3. @keyframes

We will start creating our CSS animation by defining it's keyframes using the **@keyframes** rule. **@keyframes** are essentially a list describing everything that should happen over the course of the animation. We define the values for the animating properties at various points during the animation and any property that we will see change over the course of one's cycle of an animation.

- The **@keyframes** rule works like most other CSS rules, anything contained within its curly braces are considered part of that block.

- The second step to our **@keyframes** rule is giving our animation a name. It's really important to name the animation because without a name, it would not be able to assign the animation to an element.

There are a few options available to us for how we can define each **@keyframes** within our **@keyframes** rule:

- **from, to**: You define where to animate from and where to animate to.

3.9.4. CSS animation using transition and keyframe

Example 25) create an animation that will move an image from one place to another within the x-axis.

For this exercise, we will download two images, one for the moving object and the other one as a back image. Once we have the images, will be create two containers for each of the images using with position relative and absolute respectively.

HTML code

```
<h3>CSS animation using @keyframes with attributes from
- to </h3>
<section class="mountainImage"><img src="mountain.jpg"/>
    <div class="busImage"><img src="bus.png"/></div>
</section>
```

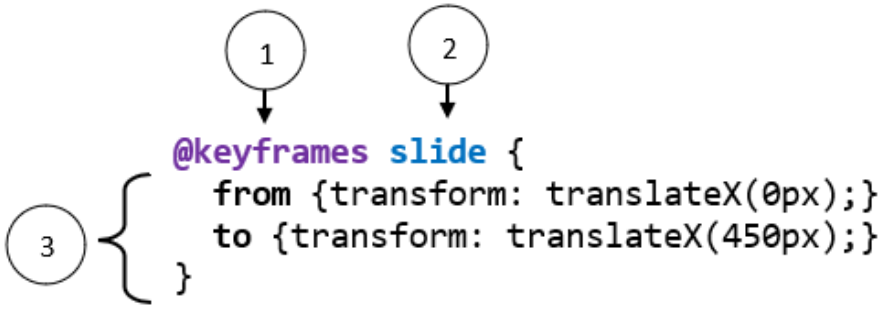
CSS code

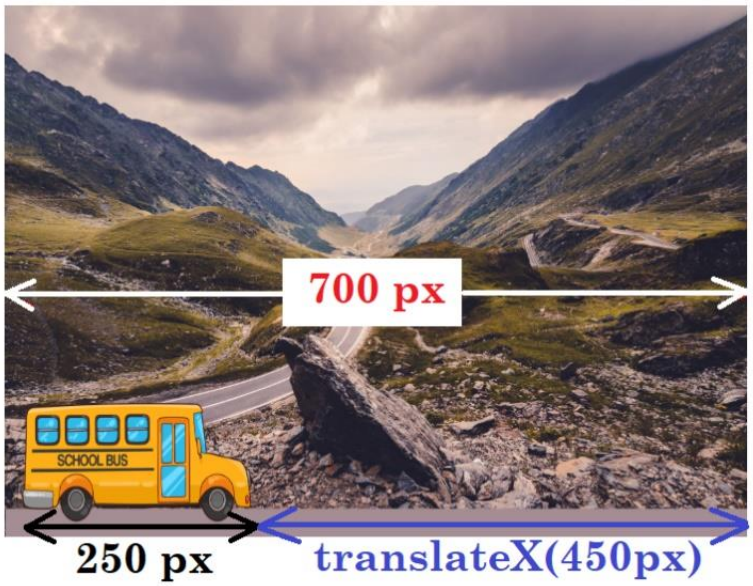
```
.mountainImage{
  width: 700px;
  height: 500px;
  position: relative;
  margin: auto;}
```

```
.busImage{
  width: 250px;
  height: 150px;
  position: absolute;
  bottom: 0;}
```

Once we have the images set up, we can start writing the css animation using @keyframes. We are going to name this animation *slide*. The animation will make the element to move from left to right. The basic rules to use @keyframes are:

1. Use @keyframes to create the animation frame
2. Assign a name to the keyframe
3. Specify what do you want the animation to do. For this project, we want the image to move *from* left *to* right. For this, we need to transform the element within the X axis from position 0 to 220px.





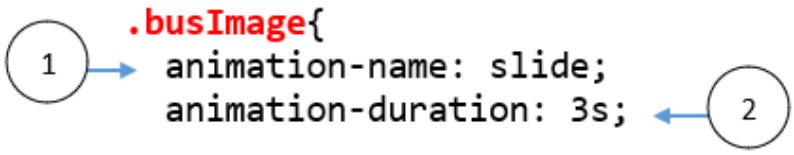
Now that we have the animation created, we need to add it into the image that we want to be animated, in our case, we will add the animation to the element that has the bus image, which has the class name ***busImage***:

```
div class="busImage"><img src="bus.png"/></div>
```

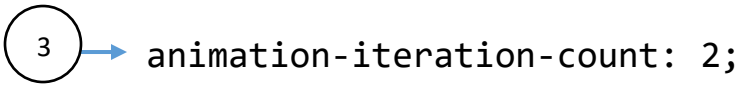
Now in css, we call for the class name and add some animation properties to it:

1. **animation-name: slide;** ➔ This tells our image with class name **busImage** to apply the **@keyframes** animation named **slide**

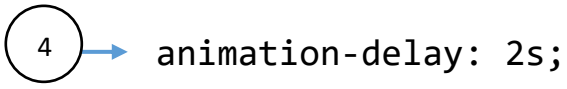
2. **animation-duration: 2s;** ➔ The second property we need to define is the animation duration. Our keyframes define what should change over the course of an animation, but they don't give any indication as to how long this should take, so that's what we do in animation duration. For this case, our animation **slide** is set to 3 seconds.



3. The **animation-iteration-count** property is also a good one to set for yourself even though it has a default value. This one determines how many times the animation will play. By default, the animation runs once, but we can set it runs **infinite** times.

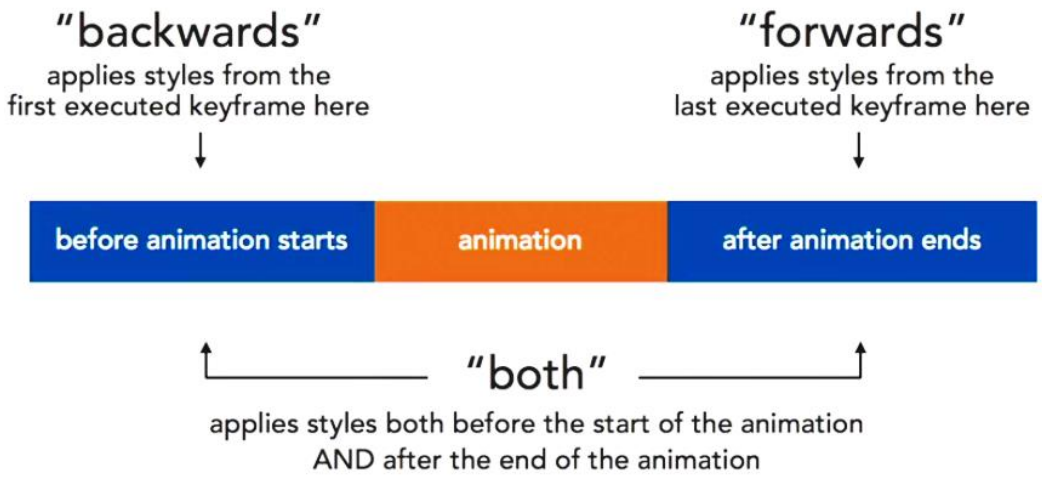


4. **animation-delay** property sets the waiting before the animation executes.



5. **animation-fill-mode** is a way of telling the animating element what to do outside of the actual duration of its animation and it can take four values: *none, backwards, forwards, and both*. The default value is *none*. If we use

the value *forwards*, the image will stay at the position at the end of the animation.



5 → `animation-fill-mode:`

6. **animation-direction** let us to manipulate what order our **keyframes** are executed in. Animation-direction can be set to *normal, reverse, alternate, and alternate-reverse.*

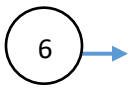
   An animation-direction of **normal** means that all iterations of the animation will be played as specified. So your keyframes will play from start to end. That means your keyframes will play from your from keyframe to your to keyframe, or your 0% percent keyframe to your 100% keyframe exactly in the order that you wrote them.

   The animation direction of **reverse** means that all iterations of your animation will be played in the reverse direction. Your last keyframe will be played first, and your first keyframe will be played last. So this would play your animation from your to keyframe to your from keyframe, or your 100% keyframe to your 0% keyframe.

   An animation-direction of **alternate** means that the direction of your animation will alternate each iteration of the animation. It will play in the normal direction the first time. So that would be 0% keyframe to 100% keyframe, and in the reverse direction the second time. So that would be the 100% keyframe back to 0%. **alternate** can only be used if your animation

plays more than once because otherwise you won't have any space to see the alternating. So we'll change our animation iteration count to two. With those two small changes made, we'll go back to our robot and see how it's changed.

An **alternate-reverse** works in very much the same way as alternate, but it starts playing your animation in the reverse direction first, and then normal. **alternate-reverse** works in much the same way as *alternate,* it plays in reverse the first time, and then forwards, but it still plays twice.




```
animation-direction: alternate;
```

animation-timing-function is the way speed is distributed across the duration of our animation. The animation-timing-function keywords include *ease, linear, ease-in, ease-out, and ease-in-out*.

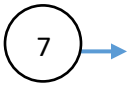
With the **linear** easing our robot moves across the screen at the same constant speed for the entire animation. Linear easing creates a constant speed of motion that never changes at all. This is often perceived as mechanical motion because nothing in real life actually moves like that.

ease allows the animation to start and slowly and then speed up to a uniform speed.

ease-in has a distinct starting slow and then speeding up as the image reaches it's destination.

ease-out is the opposite of ease-in. The image starts at a higher speed and then slows down as it gets to it's destination.

ease-in-out combines the effects of ease-in and ease-out. It starts out a little bit slow, hits a top speed in the middle of the animation, and then slows down as it reaches the end.




```
animation-timing-function: ease;
```

Infinitely looping animation

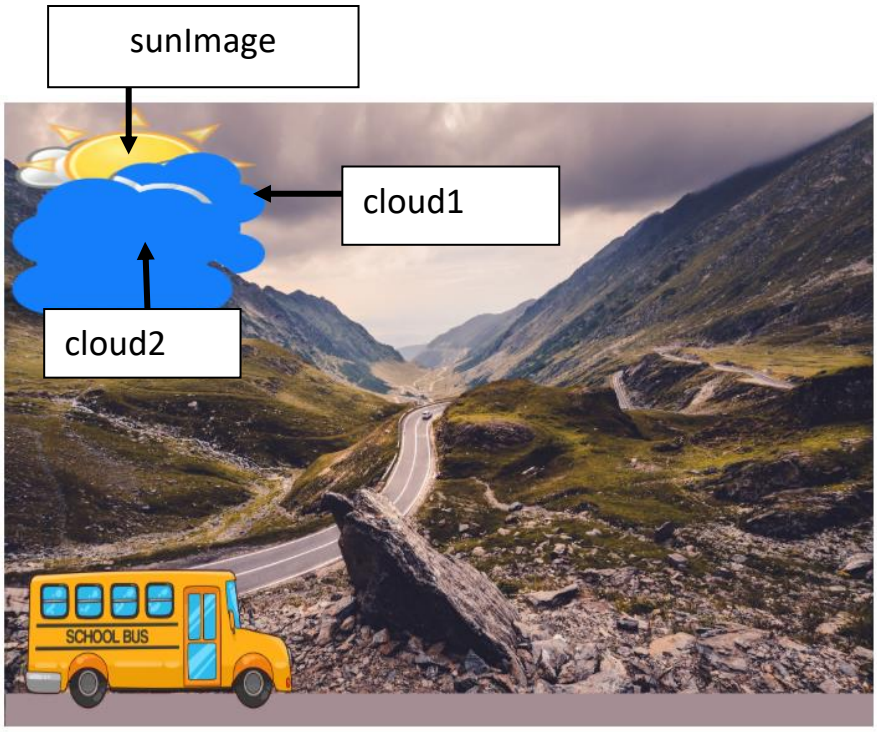
CSS is great for looping animation because it makes it possible to set an infinite loop with much less code than other methods.

Example 25a) Using the previous exercises, creating a couple of animated clouds that continently drift across the sky with different distances and speed. For this, we can download a cloud image and create three containers of different sizes for each cloud.

HTML code

```
<div class="mountainImage"><img src="mountain.jpg"/>
      <div class="busImage"><img src="bus.png"></div>
      <div class="sunImage"><img src="sun.png"></div>
      <div class="cloud1"><img src="cloud1.png"></div>
      <div class="cloud2"><img src="cloud1.png"></div>
</div>
```

Since the clouds are displayed on top of the back image, the containers' position must be set to absolute. We can also set different width and height to the clouds container and also the top position to each cloud.



CSS code

```
/* animation clouds */
.sunImage{
  width: 30%;
  height: 20%;
  position: absolute;
  top: 0;
}
.cloud1{
  width: 30%;
  height: 26%;
  position: absolute;
  top: 20px;
}
.cloud2{
  width: 30%;
  height: 26%;
  position: absolute;
  top: 60px;
}
```

Once we have the HTML and CSS set, now we can set the animation using the @keyframes. Since we want the clouds to flow from left to right, we can use the property **from** to **to**

CSS code

```
@keyframes cloudsFlowing {
  from {transform: translateX(0);}
  to {transform: translateX(500px);}
}
```

We are going to use the same animation for all three clouds, but we are going to adjust the properties to make each cloud to behave in a slightly different way. For example, we can set different **animation-iteration-count** to each cloud.

CSS code

```
. sunImage{
  width: 30%;
  height: 20%;
  position: absolute;
  top: 0;
  animation-name: cloudsFlowing;
  animation-duration: 30s;
  animation-timing-function: linear;
  animation-iteration-count: infinite;
}
```

The animation properties can also be written in one line using the property **animation**. For example, we can simplify the following properties:

animation-name: **cloudsFlowing**;
animation-duration: **30s**;
animation-timing-function: **linear**;
animation-iteration-count: **infinite**;

using **animation** property, we can write each property values in one line and separate the values with a space:

animation: cloudsFlowing 30s linear infinite;

Now, we can apply the **cloudsFlowing** animation to the other two clouds with 8 and 5 seconds respectively.

CSS code

```
.cloud1{
  width: 30%;
  height: 26%;
  position: absolute;
  top: 20px;
  animation: cloudsFlowing 10s linear infinite;
}
.cloud2{
  width: 30%;
  height: 26%;
```

```
  position: absolute;
  top: 60px;
  animation: cloudsFlowing 6s linear infinite;
}
```

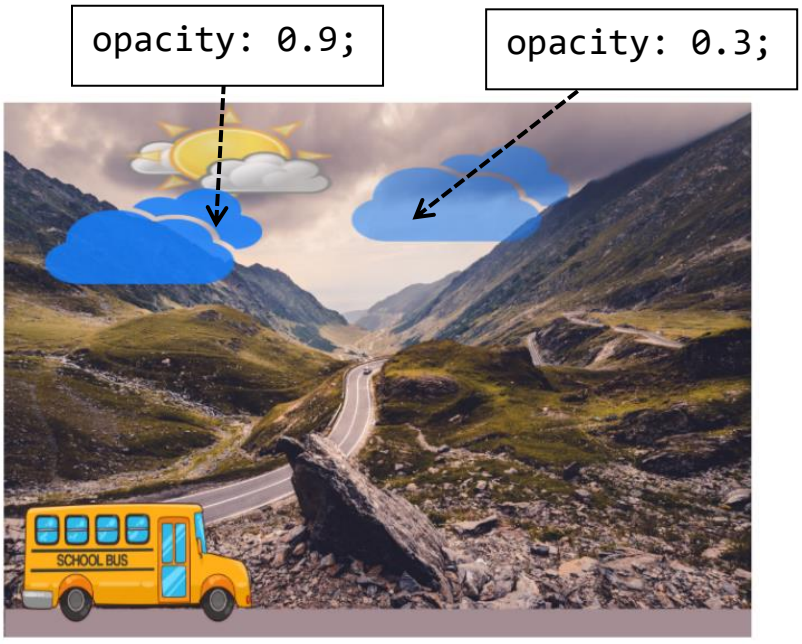
opacity

One of the transition properties is **opacity**. **Opacity** property happens when a new value is assigned to it. It will cause a smooth change between the old value and the new value over a period of time. The value of opacity is ranged in between 0 and 1 where 0 is clear and 1 is the full color of the element.

Example 25b) Using the previous animation, make the clouds to be almost half visible when they reach the edge of the right side.

CSS code

```
@keyframes cloudsFlowing {
  from{transform: translateX(0); opacity: 0.9;}
  to{transform: translateX(500px); opacity: 0.3;}
}
```



CSS animation using "%" time frame

CSS animation in an element can be slip using % time frame. For example, from a **@keyframes** animation using **from** to **to** attributes, we can replace **from** to **0%** and **to** to **100%**:

```
@keyframes cloudsFlowing{
  from{ transform: translateX(0px); opacity:1;}
  to{transform: translateX(250px);opacity:0.5;}
}

/* The same effect as the keyframes above */
@keyframes cloudsFlowing{
  0%{ transform: translateX(0px); opacity:1;}
  100%{transform: translateX(250px);opacity:0.5;}
}
```

Example 26) Create an animation that will move a square from one position to another and change its background color at the different position. The total time of the animation is 10s and the animation will repeat 20 times.

To understand how the position works on an element, first we need to create a container to place the square:

HTML code

```
<h3> CSS animation using "%" time frame </h3>
<div class=" frameContainer">
</div>
```

CSS code

```
.frameContainer{
  width: 1500px;
  height: 600px;
  background-color: lightblue;
  position: relative;
}
```

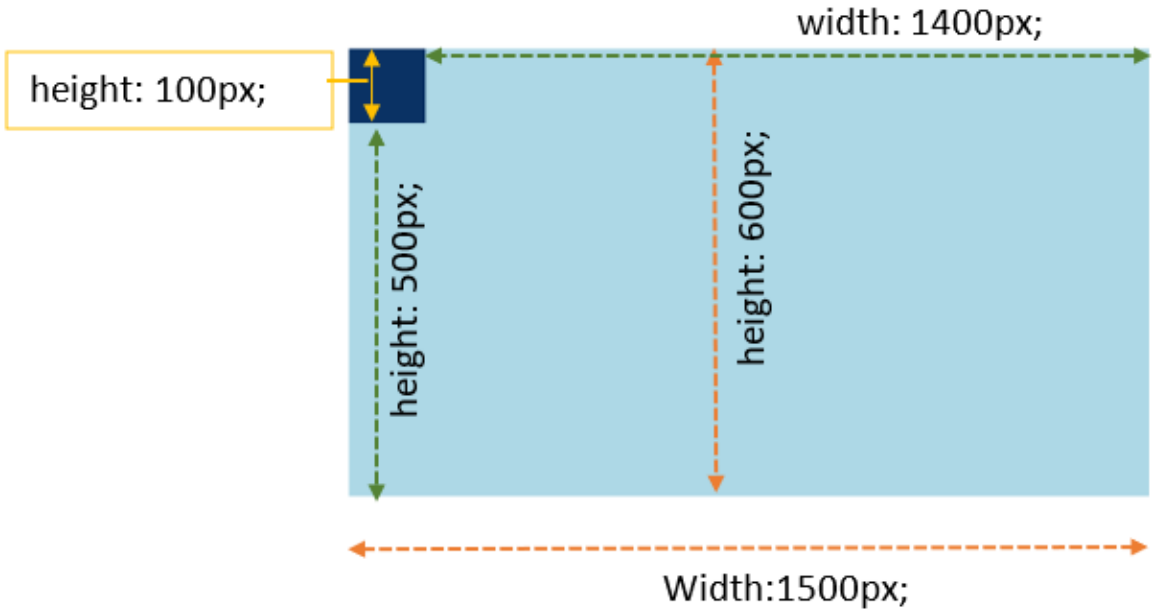
Once we have the container for the square, we create the square that sits inside the container:

HTML code

```
<div class="frameContainer">
      <div class="animationSquare"></div>
</div>
```

CSS code

```
.animationSquare{
    width: 100px;
    height: 100px;
    background-color: rgb(10,50,100);
    position: absolute;
}
```

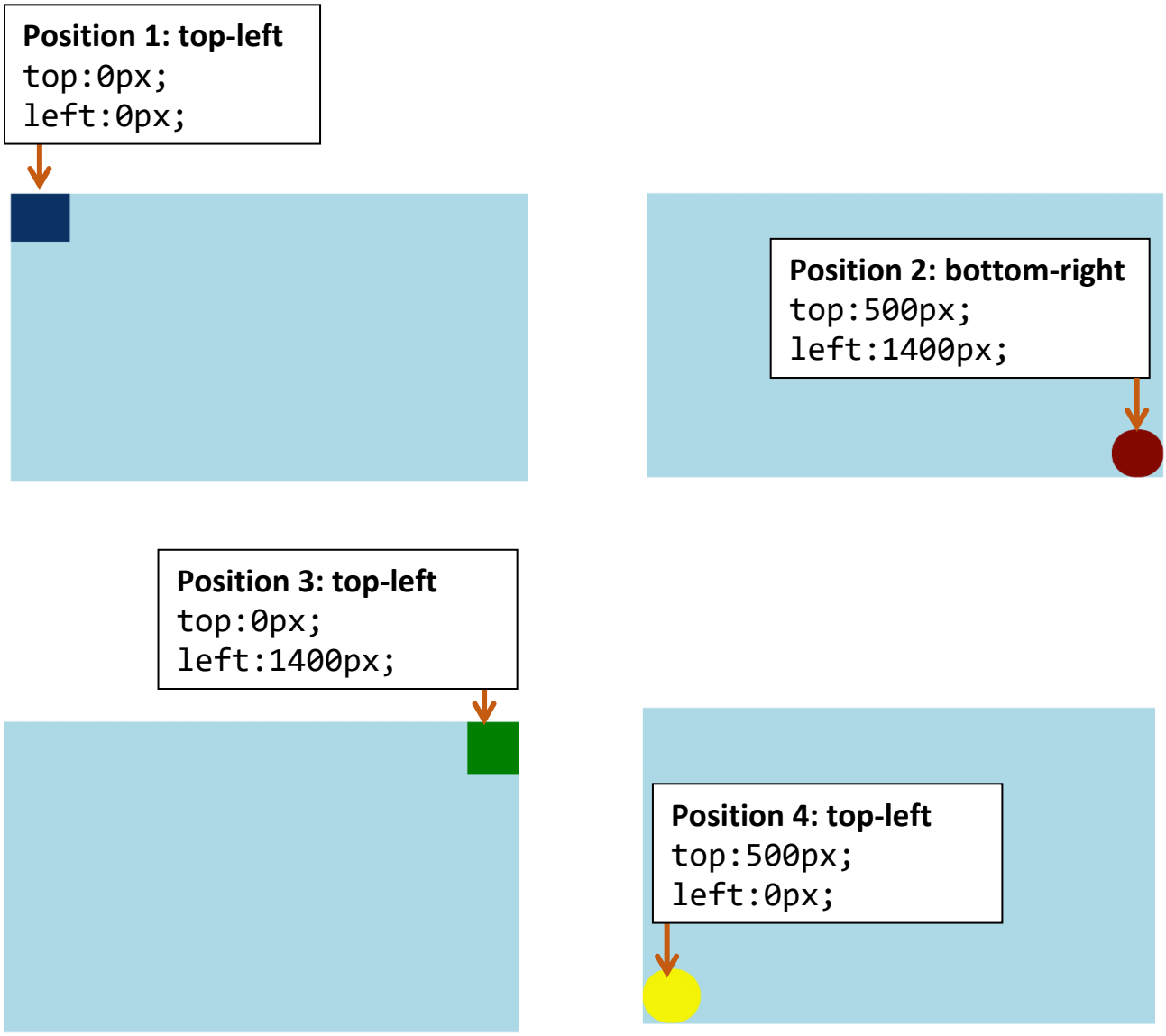


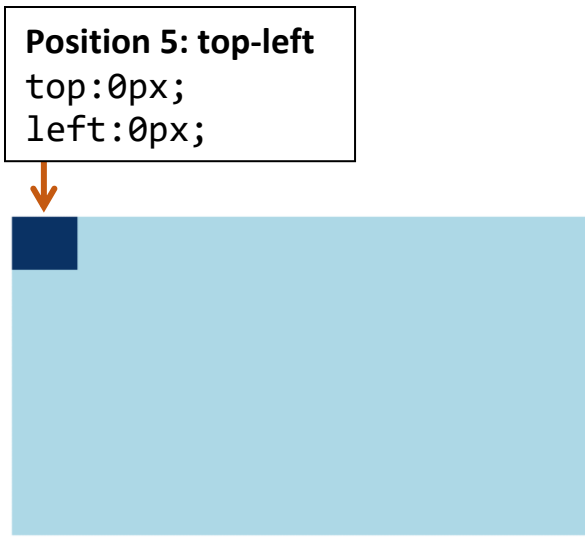
After the square is set, we need to create the animation using % time frame as the following:

Total time frame: 10 seconds

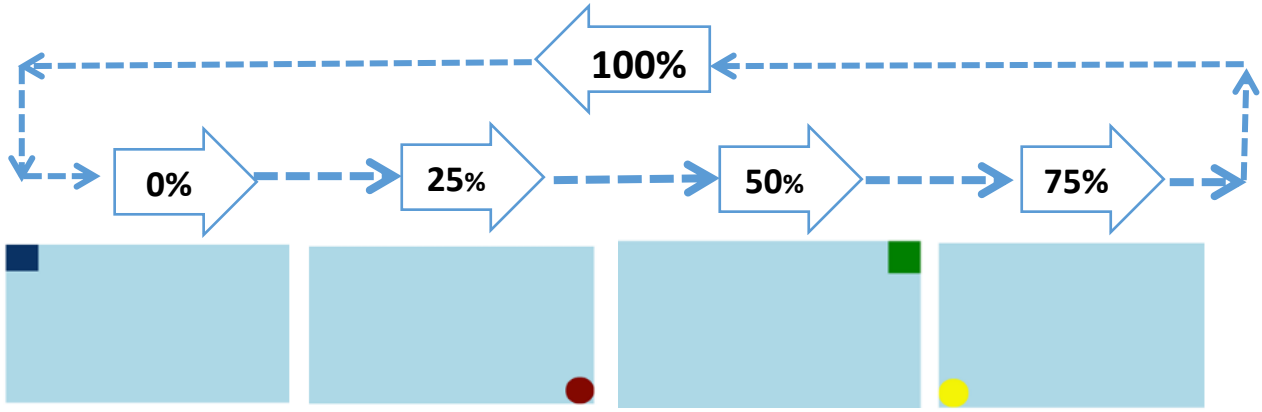
- Position 1:
    - time frame: 0% (0 to 2 seconds)
    - background color: darkblue
    - border radius: 0%(squared shape)
    - position: top-left
- Position 2:
    - time frame: 25% (2.1 to 4 seconds)
    - background color: darkred
    - border radius: 50%(circle shape)
    - position: bottom-right
- Position 3:
    - time frame: 50% (4.1 to 6 seconds)
    - background color: green
    - border radius: 0%(squared shape)
    - position: top-right
- Position 4:
    - time frame: 75% (6.1 to 8 seconds)
    - background color: yellow
    - border radius: 50%(circle shape)
    - position: bottom-left
- Position 5:
    - time frame: 100% (8.1 to 10 seconds)
    - background color: darkblue
    - border radius: 0%(squared shape)
    - position: top-left

To create the animation, we need to understand the position first. The position of an element is measured from the top-left corner of the element:


Position 1: top-left
top:0px;
left:0px;
Position 2: bottom-right
top:500px;
left:1400px;
Position 3: top-left
top:0px;
left:1400px;
Position 4: top-left
top:500px;
left:0px;



One complete cycle of the animation:



CSS code

```
@keyframes animation_square{
0% {background-color: darkblue; left: 0px; top: 0px;
border-radius: 0%;}
  25%{background-color: darkred; left:1400px; top:
500px; border-radius: 50%;}
  50%{background-color: green; left: 1400px; top: 0px;
border-radius: 0%;}
  75%{background-color: yellow; left: 0px; top: 500px;
border-radius: 50%;}
  100%{background-color: darkblue; left: 0px; top:
0px;border-radius: 0%;}
}
```

Rotating animation

Rotation is a CSS property that defines a 2D rotation and the rotation angle is specified within the parenthesis. For example, if we want to rotate an element 200 degree clockwise, we can write: rotate(200deg). If we want to rotate an element 200 degree counter-clockwise, we can write: rotate(-200deg)

Example 27) Create an animation that will rotate an image clockwise 180 degree and then 360 degree counter-clockwise. The animation duration will be 6s and it will repeat 10 times.

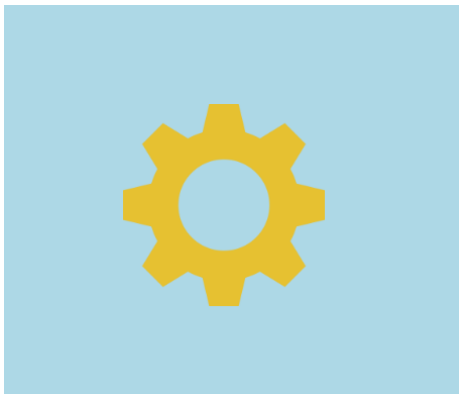
For this animation, first, we create the animation structure in HTML and CSS:

HTML code

```
<div class="frameRotation">
   <div class="rotationImage">
        <img src="hexagon.png"/>
   </div>
</div>
```

CSS code

```
.frameRotation{
  width: 1500px;
  height: 500px;
  background-color: lightblue;
  position: relative;}
.rotationImage{
width: 200px;
  height: 200px;
  position: absolute;
  top: 20%;
  left: 40%;
  animation: spin 10s 20;
}
```



Once the structure is set, we can start the animation according to the following time frame:

Total time frame: 6 seconds

- time frame: 0%
  - rotate 0 degree
- time frame: 30%
  - rotate 180 degree clockwise
- time frame: 70%
  - rotate 360 degree clockwise
- time frame: 100%
  - rotate 0 degree

CSS code

```
@keyframes spin {
  0%{transform: rotate(0deg);}
  30%{transform: rotate(180deg);}
  70%{transform: rotate(-360deg);}
  100%{transform: rotate(0deg);}
}
```

Zooming an image

CSS animation transform property has value scale( ) to create zoom-in and zoom-out effect on an element. There are different parameters that we can use within the scale's parenthesis, for now, we are going to use one value inside the parenthesis. The highest value is 1 and the lowest value is 0. 1 means 100% of the actual element's size, and 0 means 0% of the actual element's size.

100% of the element size
transform: scale(1);



30% of the element size
transform: scale(0.3);



Exercise 6: CSS Animation Exercise: Interactive Card Effects
Objective

Create an interactive card layout that demonstrates:

- CSS transitions (smooth hover effects)
- CSS transforms (scale, rotate, translate)
- CSS keyframe animations (continuous movement or attention effects)

Instructions

Create a file named:

- animation_practice.html
- animation_style.css

Then complete the following exercises:

Exercise a: Hover Transition Card

Task:

- Create a card that smoothly changes appearance when hovered.

Requirements:

- Add a
- Apply a background color, padding, and border-radius
- Use transition to animate:
  - background color change
  - box-shadow increase
  - slight scaling effect

Example Behavior:

When hovered:

- Card becomes slightly larger
- Shadow becomes deeper
- Background color changes smoothly

Exercise b: Transform Effects Button

Task:

- Create a button that uses CSS transforms.

Requirements:

- Add a <button class="transform-btn">Click Me</button>
- On hover:
    - Rotate slightly (rotate)
    - Move upward (translateY)
    - Scale up slightly (scale)
- Use transition to make it smooth

Example Behavior:

Button “lifts” and tilts when hovered.

Exercise c: Keyframe Animation Loader

Task:

- Create a loading indicator using keyframes.

Requirements:

- Add a
- Style it as a circle
- Use @keyframes to:
    - Rotate continuously (360°)
    - OR pulse (scale up and down)

Example Behavior:

- Spinner rotates endlessly OR pulses like a heartbeat

Exercise d: Animated Notification Badge

Task:

- Create a notification icon with attention-grabbing animation.

Requirements:

- Add a `<div class="notification">3</div>`
- Use @keyframes to:
    - Make it “shake” slightly or bounce
- Add hover effect using transform: `scale()`

Example Behavior:

- Badge shakes every few seconds
- Grows slightly when hovered

Bonus Challenge

Combine all effects:

- Card contains a button
- Button triggers animation on the card
- Loader appears inside the card on hover

Skills Practiced

- transition
- transform (scale, rotate, translate)
- @keyframes
- animation-duration, animation-iteration-count
- UI interaction design

3.10. Responsive Webpages

Making a webpage responsive to the screen size is an important features in web technology. Nowadays, web pages open on devices with different screen sizes, for example a **desktop computer** screen has a width of 1200px, a Samsung Galaxy Tab 10.1 tablet has the width of 900px, and the width of a Samsung

Galaxy J7 smartphone is 720px . Therefore, it is important that when you design a webpage, the elements on the webpage can adjust to the screen of the device. You can visit: http://screensiz.es/ to view more devices width.

For app view development, it is an important to add the following line:

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

The **viewport** is the user's visible area of a web page. It varies with the device, and will be smaller on a mobile phone than on a computer screen. **viewport** element gives the browser instructions on how to control the page's dimensions and scaling.

The **width=device-width** part sets the width of the page to follow the screen-width of the device (which will vary depending on the device).

The **initial-scale=1.0** part sets the initial zoom level when the page is first loaded by the browser.



Without the viewport meta tag



With the viewport meta tag

3.10.1. @media query

Media query is a CSS technique introduced in CSS3 and it is used to make responsive pages.

It uses the @media rule to include a block of CSS properties only if a certain condition is true.

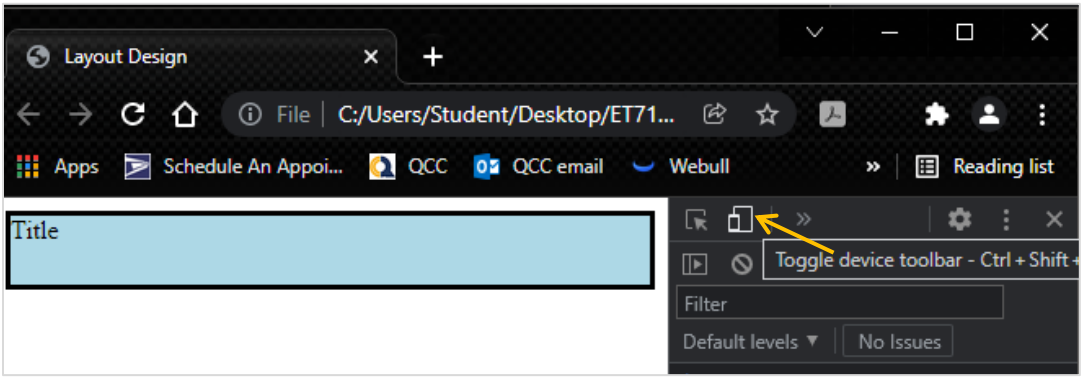
When using @media, instead of changing styles when the width gets *smaller* than 800px, we should change the design when the width gets *larger* than 800px. This will make our design Mobile First. The syntax code will look as:

```
@media only screen and (min-width: 800px){
}
```

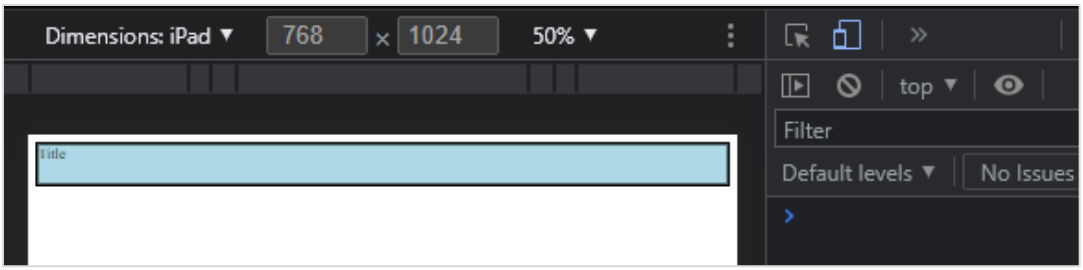
Between the curly brackets should go the CSS attributes of the elements that will changed when the screen has the width of 800px or greater.

Mobile First means designing for mobile before designing for desktop or any other device (This will make the page display faster on smaller devices). Some web developer prepares to design a mobile view first as it moves toward the tablet's, laptop's, and desktop's screen size. Therefore, when we apply @media query, the screen size has property **min-width: 800px**. On the other hand, since the material in this lab manual was designed from a desktop computer screen view, then we can design from the desktop computer screen toward the tablet's and smartphone's screen size. For this, instead of using **min-width:800px** we use **max-width: 800px**

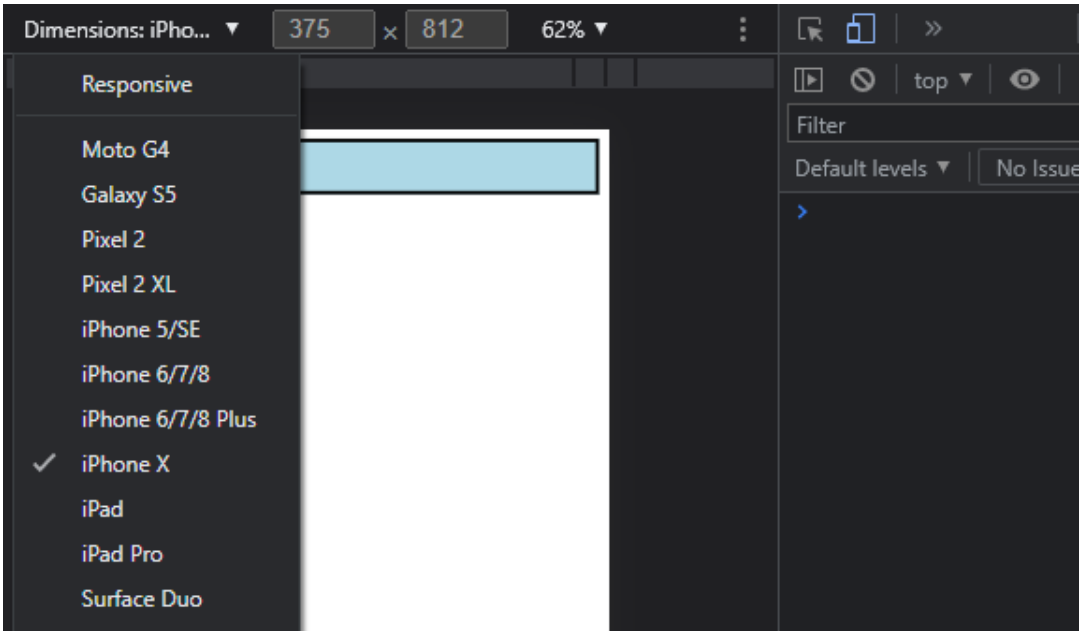
Also, if we are designing from mobile view first, we can change the browser view to mobile view by using the function key **F12** and then click on the **Toggle device toolbar**



Once clicked on the Toggle device toolbar, we can select the size of the mobile view:



Let us to pick the iPhone X screen size. In this case, since iPhone X has a width of 375px, then we can design a view up to 375px or 450px so the design can be used to other smartphone screen.

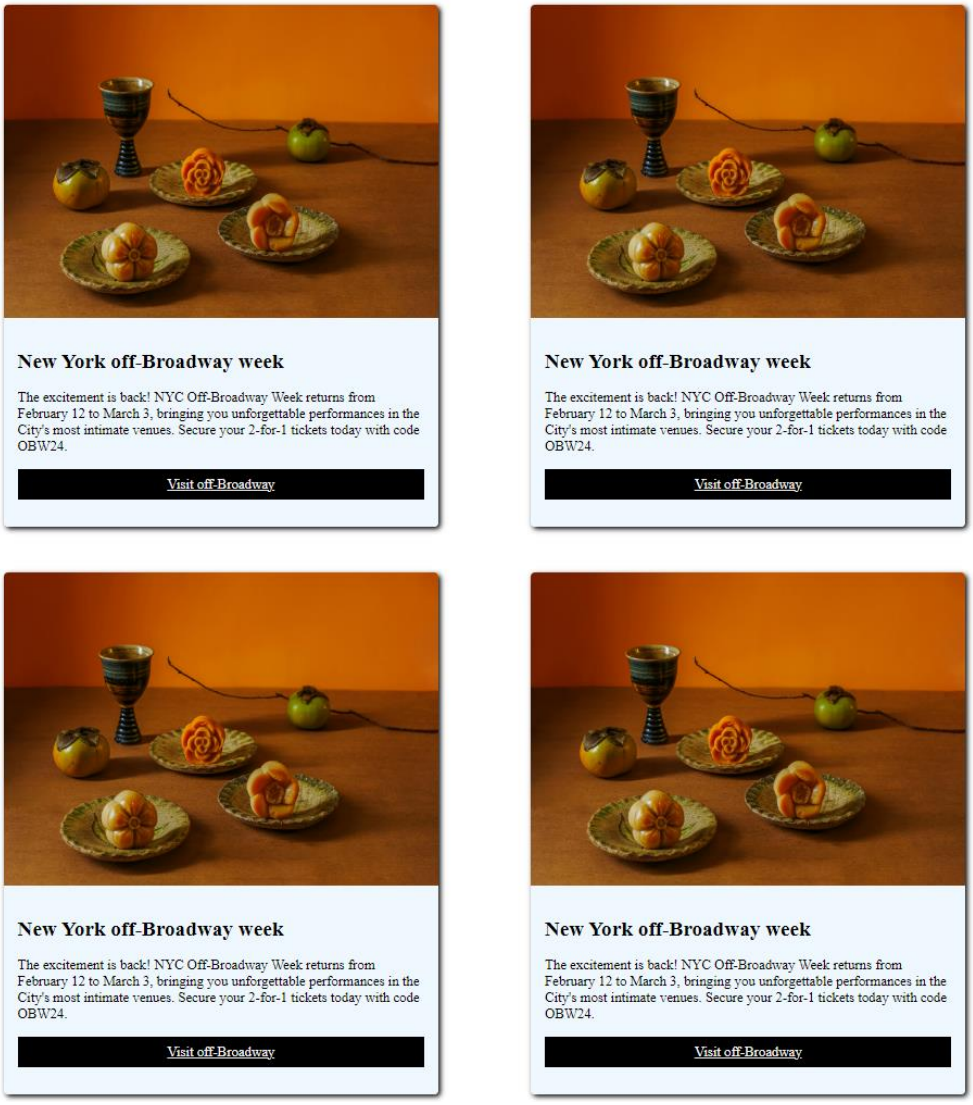


Example 29) create four cards webpage and apply responsive properties to the cards. use media query to adjust the CSS styling as:

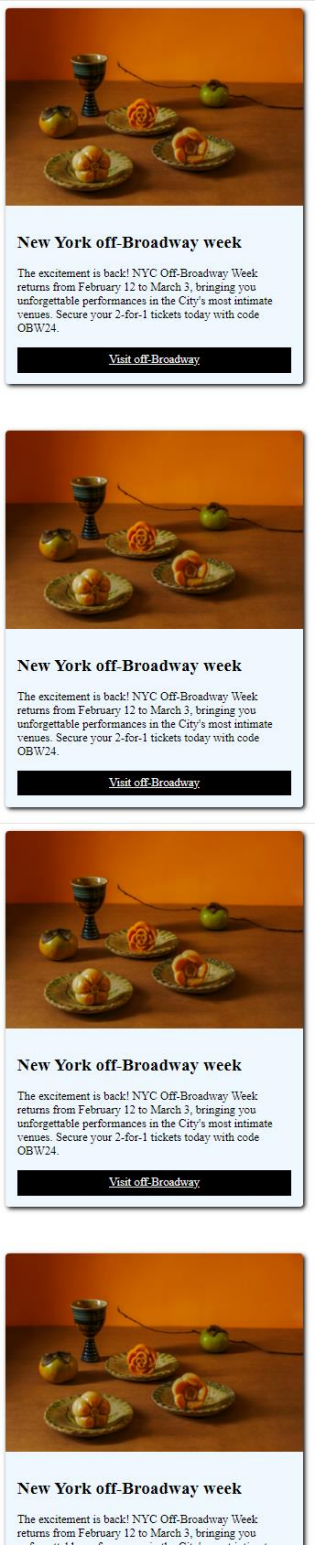
- At 1700px, make the browser view to change two cards. To do so, adjust the width of the cards.
- At 1450px, make the browser to view one card.



The browser view smaller than 1700px would look as:



The browser view smaller than 1400px would look as:



HTML code

```
<main class="maincontainer">
    <!-- card 1 -->
    <section class="card">
        <img src="images/fruits.jpg" class="imgcard">
        <div class="cardinfo">
           <h2>New York off-Broadway week</h2>
           <p>The excitement is back! NYC Off-Broadway
           Week returns from February 12 to March 3,
           bringing you unforgettable performances in
           the City's most intimate venues. Secure your
           2-for-1 tickets today with code OBW24.
           </p>
           <a href="https://ww.nyctourism.com/off-
           broadway-week/" target="_lank" rel="noopener
           noreferrer" class="cardlink">Visit off-
           Broadway</a>
        </div>
     </section>

     <!-- card 2 -->
     <section class="card">
        <img src="images/fruits.jpg" class="imgcard">
        <div class="cardinfo">
           <h2>New York off-Broadway week</h2>
           <p>The excitement is back! NYC Off-Broadway
           Week returns from February 12 to March 3,
           bringing you unforgettable performances in
           the City's most intimate venues. Secure your
           2-for-1 tickets today with code OBW24.
           </p>
           <a href="https://ww.nyctourism.com/off-
           broadway-week/" target="_lank" rel="noopener
           noreferrer" class="cardlink">Visit off-
           Broadway</a>
        </div>
     </section>
```

```
    <!-- card 3 -->
    <section class="card">
        <img src="images/fruits.jpg" class="imgcard">
        <div class="cardinfo">
           <h2>New York off-Broadway week</h2>
           <p>The excitement is back! NYC Off-Broadway
           Week returns from February 12 to March 3,
           bringing you unforgettable performances in
           the City's most intimate venues. Secure your
           2-for-1 tickets today with code OBW24.
           </p>
           <a href="https://ww.nyctourism.com/off-
           broadway-week/" target="_lank" rel="noopener
           noreferrer" class="cardlink">Visit off-
           Broadway</a>
        </div>
     </section>
</main>
```

CSS code

```
main.maincontainer{
    width: 80%;
    margin: 10%;
    display: flex;
    justify-content: space-evenly;
    flex-wrap: wrap;
}
section.card{
    width: 300px;
    height: 600px;
    box-shadow: 2px 2px 6px black;
    border-radius: 5px;
    overflow: hidden;
    margin: 1em;
}
img.imgcard{
    width: 100%;
    height: 40%;
```

```
    display: block;
}
div.cardinfo{
    background-color: aliceblue;
    padding: 1em;
    height: 60%;
}
a.cardlink{
    display: inline-block;
    background-color: black;
    color: whitesmoke;
    padding: 0.5em;
    width: 100%;
    text-align: center;
    transition: 300ms;
}
a.cardlink:hover{
    background-color: crimson;
    box-shadow: 0px 0px 6px 2px black;
    transform: scale(1.01);
    font-size: 1.05em;
    text-decoration: none;
}

/*------ BREAKPOINT = 1700PX ---------*/
@media only screen and (max-width:1700px){
    section.card{width: 500px ; margin: 2em;}
    img.imgcard{height: 60%;}
}
/*------ BREAKPOINT = 1400PX ---------*/
@media only screen and (max-width: 1400px){
    section.card{width:100%; height: auto;}
    img.imgcard{height: auto;}
    img.cardinfo{height:auto;}
}
```

3.10.2. Using CSS Grid to Build Modern Layouts and Responsive Web Pages

This topic explores how to use CSS Grid to create structured, flexible layouts that automatically adapt to different screen sizes. It covers defining grid containers and items, organizing content into rows and columns, and using responsive techniques such as fractional units, media queries, and auto-fit/auto-fill to ensure a seamless user experience across desktop, tablet, and mobile devices.

Basic CSS Grid Layout (Simple 2x2 Grid)

HTML code

```
<div class="grid-container">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
  <div>Item 4</div>
</div>
```

CSS code

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: auto;
  gap: 10px;
}

.grid-container div {
  background: lightblue;
  padding: 20px;
  text-align: center;
}
```

Responsive Grid Using auto-fit
This layout automatically adjusts based on screen size.

HTML code

```
<div class="responsive-grid">
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
  <div>Box 4</div>
  <div>Box 5</div>
</div>
```

CSS code

```
.responsive-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px,
1fr));
  gap: 15px;
}

.responsive-grid div {
  background: lightcoral;
  padding: 20px;
  text-align: center;
}
```

This makes items stack on small screens and expand on larger screens.

Layout Using Grid Template Areas (Website Structure)

```
<div class="page">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main Content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</div>
```

CSS code

```
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "nav main"
    "nav aside"
    "footer footer";
  grid-template-columns: 200px 1fr;
  gap: 10px;
}


header { grid-area: header; background: #ccc; }
nav    { grid-area: nav; background: #eee; }
main   { grid-area: main; background: #ddd; }
aside  { grid-area: aside; background: #bbb; }
footer { grid-area: footer; background: #aaa; padding:
10px; }
```

Mobile Responsive Version (Media Query)

CSS code

```
@media (max-width: 600px) {
  .page {
    grid-template-areas:
      "header"
      "nav"
      "main"
      "aside"
      "footer";
    grid-template-columns: 1fr;
  }
}
```

On small screens, everything stacks vertically.

Example 30) Here is a complete mini project using CSS Grid to create a responsive Portfolio Web Page.

HTML code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>My Portfolio</title>

</head>

<body>

<div class="container">

  <header>
    <h1>John Doe</h1>
    <p>Web Developer | Designer | Programmer</p>
  </header>

  <nav>
    <a href="#">Home</a>
    <a href="#">Projects</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>

  <section class="hero">
    <h2>Welcome to My Portfolio</h2>
    <p>I build responsive and user-friendly web
    applications.</p>
  </section>

  <section class="projects">
    <div class="project-card">Project 1</div>
```

```
    <div class="project-card">Project 2</div>
    <div class="project-card">Project 3</div>
    <div class="project-card">Project 4</div>
  </section>

  <aside class="sidebar">
    <h3>Skills</h3>
    <ul>
      <li>HTML</li>
      <li>CSS Grid</li>
      <li>JavaScript</li>
      <li>Python</li>
    </ul>
  </aside>

  <footer>
    <p>© 2026 John Doe | All Rights Reserved</p>
  </footer>

</div>

</body>
</html>
```

CSS code

```
/* ====== Global Styles ====== */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

body {
  background-color: #f4f4f4;
}

/* ====== Grid Layout ====== */
```

```
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "nav nav"
    "hero hero"
    "projects sidebar"
    "footer footer";
  grid-template-columns: 3fr 1fr;
  gap: 20px;
  padding: 20px;
}

/* ====== Header ====== */
header {
  grid-area: header;
  background: #222;
  color: white;
  padding: 20px;
  text-align: center;
}

/* ====== Navigation ====== */
nav {
  grid-area: nav;
  background: #444;
  padding: 15px;
  text-align: center;
}
nav a {
  color: white;
  margin: 0 15px;
  text-decoration: none;
  font-weight: bold;
}

/* ====== Hero Section ====== */
.hero {
```

```
  grid-area: hero;
  background: #ddd;
  padding: 40px;
  text-align: center;
}

/* ====== Projects Section ====== */
.projects {
  grid-area: projects;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px,
1fr));
  gap: 15px;
}

.project-card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  text-align: center;
}

/* ====== Sidebar ====== */
.sidebar {
  grid-area: sidebar;
  background: white;
  padding: 20px;
  border-radius: 8px;
}
/* ====== Footer ====== */
footer {
  grid-area: footer;
  background: #222;
  color: white;
  text-align: center;
  padding: 15px;
}
```

```
/* ====== Responsive Layout ====== */
@media (max-width: 768px) {
  .container {
    grid-template-areas:
      "header"
      "nav"
      "hero"
      "projects"
      "sidebar"
      "footer";
    grid-template-columns: 1fr;
  }
}
```

3.11 Building Interactive Web Interfaces with CSS — Slideshows and Modal Windows

Slideshows

A slideshow allows multiple images or content panels to cycle automatically or through user interaction. Using CSS, slideshows can be built without JavaScript by leveraging techniques such as:

- **Keyframe animations (@keyframes)** to transition between slides
- **Opacity and transform properties** to create smooth fade or slide effects
- **Flexbox or grid layouts** to align slide content
- **Radio button hacks or checkbox toggles** for manual navigation

Example features include:

- Auto-playing image transitions
- Navigation dots or arrows
- Hover pause effects

CSS Modal Window

A modal window is a pop-up overlay that appears on top of the main page content, often used for alerts, forms, or image previews. CSS can control its visibility and animation using:

- **:target pseudo-class or checkbox toggle technique** for open/close behavior
- **Positioning (fixed, absolute)** to center the modal
- **Z-index layering** to ensure it appears above all content
- **Transitions and transforms** for smooth opening effects

Common modal features include:

- Darkened background overlay
- Centered content box
- Close button or click-outside-to-dismiss behavior

Learning Outcome

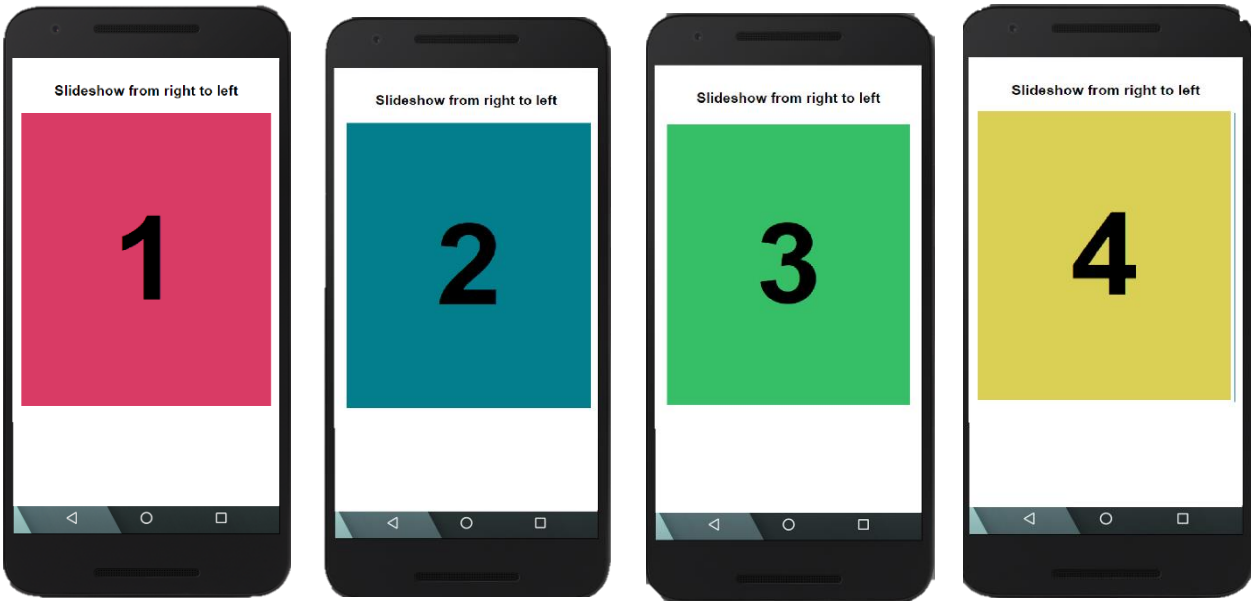
By the end of this topic, learners will be able to design responsive and interactive UI components using only CSS, enhancing user experience without relying heavily on JavaScript.

3.11.1. Creating Slideshow using CSS animation

Slideshow is one of the fun features in website. It presents to the viewer a series of information, images, or text slides looping in certain time. The purpose of a slideshow is to promote visual interest or artistic value, sometimes accompanied with description or text, or to clarify or reinforce information, ideas, comments, solutions, or suggestions.

There are different way to create slideshow, one of the way to create slideshow in CSS is to use the *animation* feature.

Example 30) Create an infinite slideshow to display a number from 1 to 4, increasing order, with different background colors.



Let's set up the html structure with a link to index.css

HTML code

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
      initial-scale=1.0">
     <link href="index.css" rel="stylesheet"
     type="text/css"/>
    <title>Slideshow</title>
  </head>
  <body>

  </body>
</html>
```

In index.css, we are going to set the **body** padding to 2%. This will prevent that elements within the body sit in the border of the smartphone's screen.

CSS code

```
body {
  font-family: Helvetica, sans-serif;
  padding: 2%;
  text-align: center;
}
*{ box-sizing: border-box; }
```

Now that we have the **body** set, we can create the slide container with the id name as **slideshow1**

HTML code

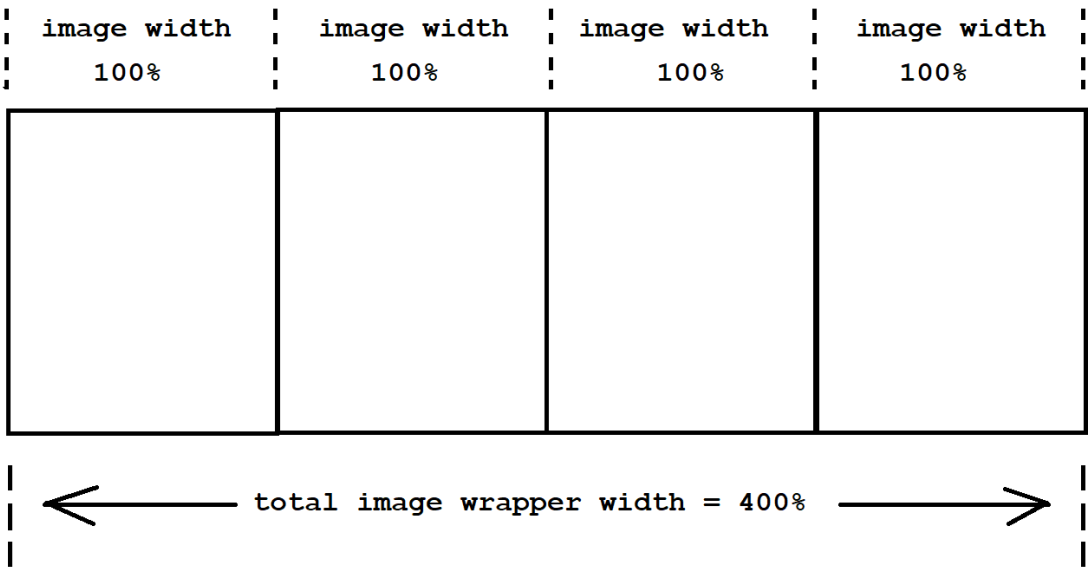
```
  <body>
      <div id="slideshow1">
      </div> <!—end of slideshow1 -->
  </body>
```

Once we have the html structure for slideshow1, in css, we are going to set the height to 400px, the width to fit 100% of the app screen, and all the overflow information in slideshow1 will be hidden. Also, we are going to add border to slideshow1 for reference, but remember that the border will be deleted once the slideshow frame is set.

CSS code

```
#slideshow1{
  overflow: hidden;
  height: 400px;
  width: 100%;
  border: solid red;
}
```

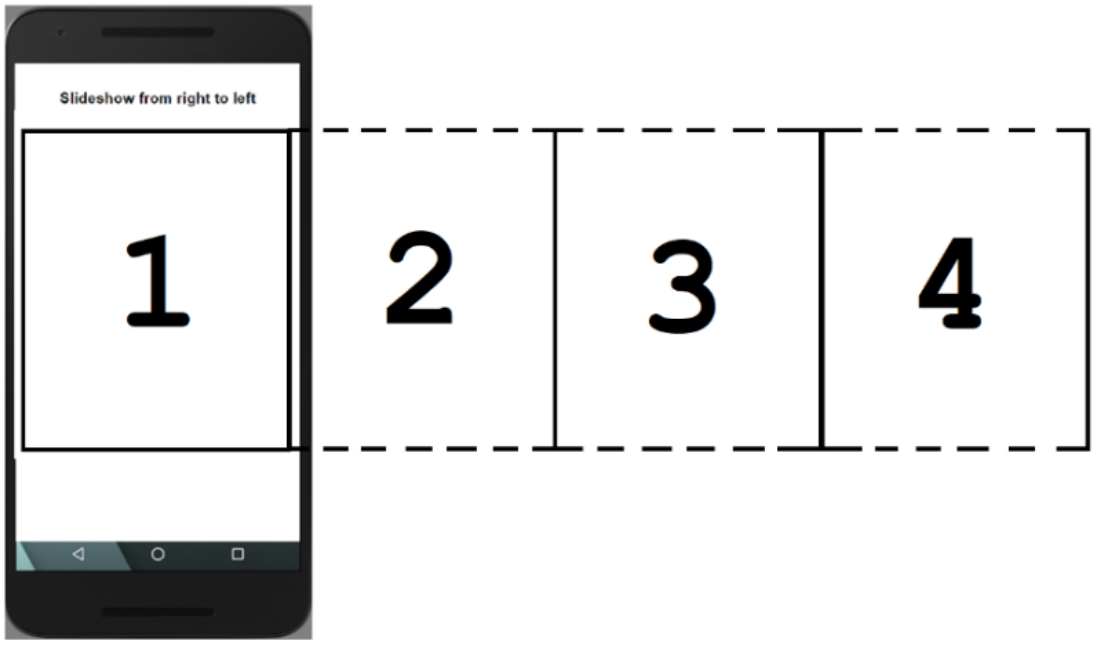
After it, we need to create a container to wrap all four numbers. We are going to name the wrapper **slide-wrapper**. The total width of the **slide-wrapper** is the sum of the width of each image in the slideshow. For example, if we are planning to display four images in a slideshow and each image will display 100% to the width of the app screen, then the width of the **slide-wrapper** will be 400%.



HTML code

```
<body>
    <div id="slideshow1">
      <div class="slide-wrapper">
      </div> <!–end of slide-wrapper -->
    </div> <!–end of slideshow1 -->
</body>
```

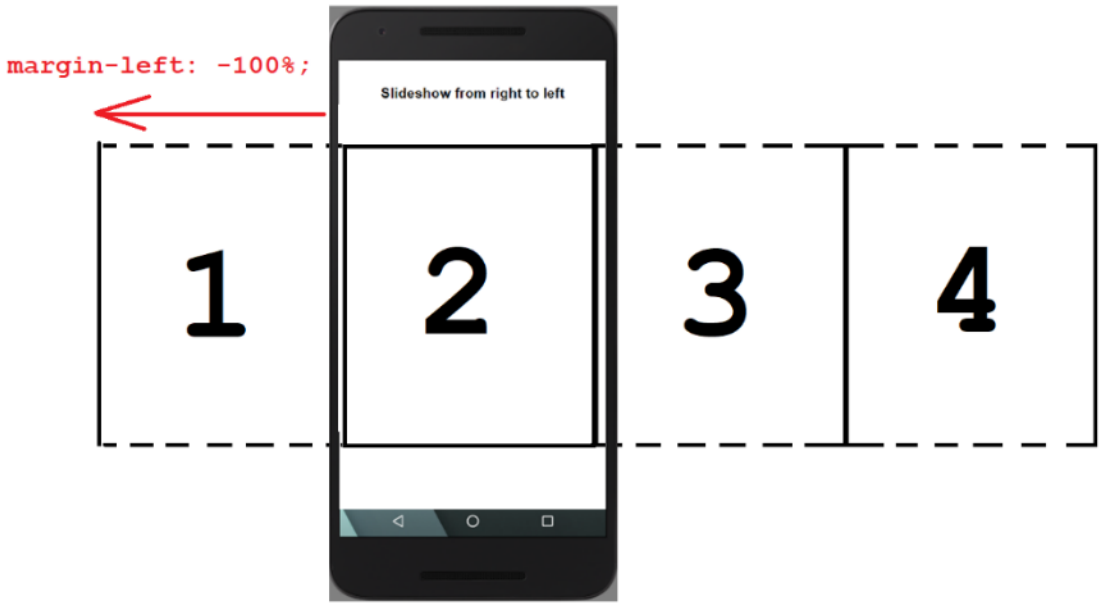
The other feature that we need to add to the **slide-wrapper** is the animation. The animation will slide each image in the slideshow from right to left one at the time. To do so, at each time frame the **slide-wrapper** will add margin to the left by 100% of the image width. Since the **slide-wrapper** is shifted to the left, the **margin-left** should be set to **-100%**. For example, at the first time frame, the slideshow will show the first image:



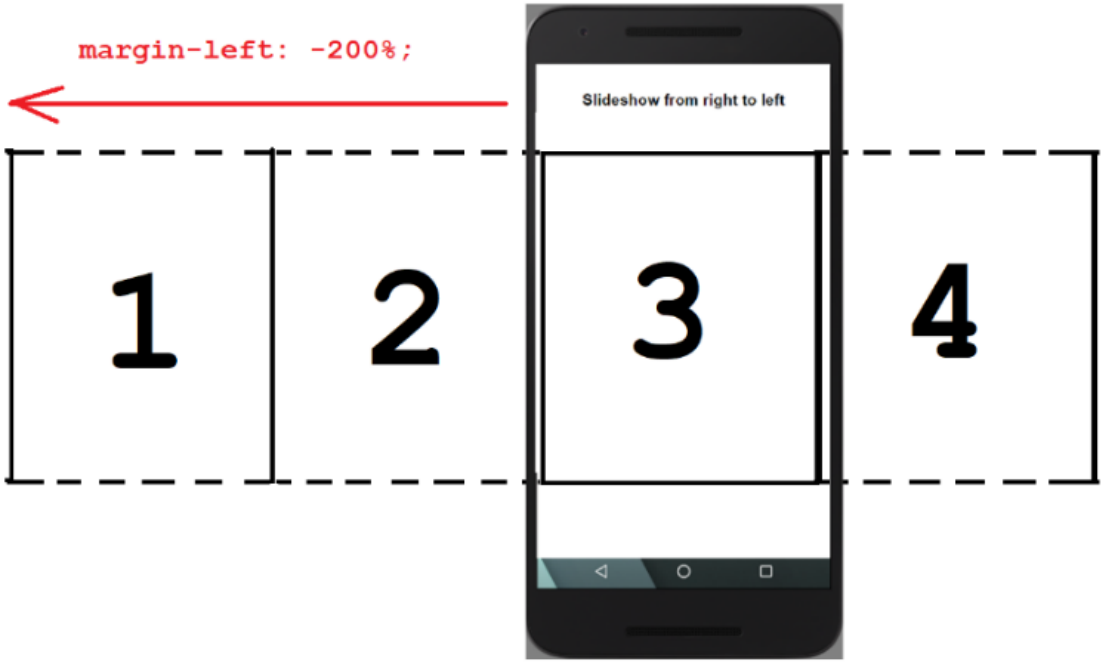
CSS code

```
.slide-wrapper {
  width: 400%;
  height: 100%;
}
```

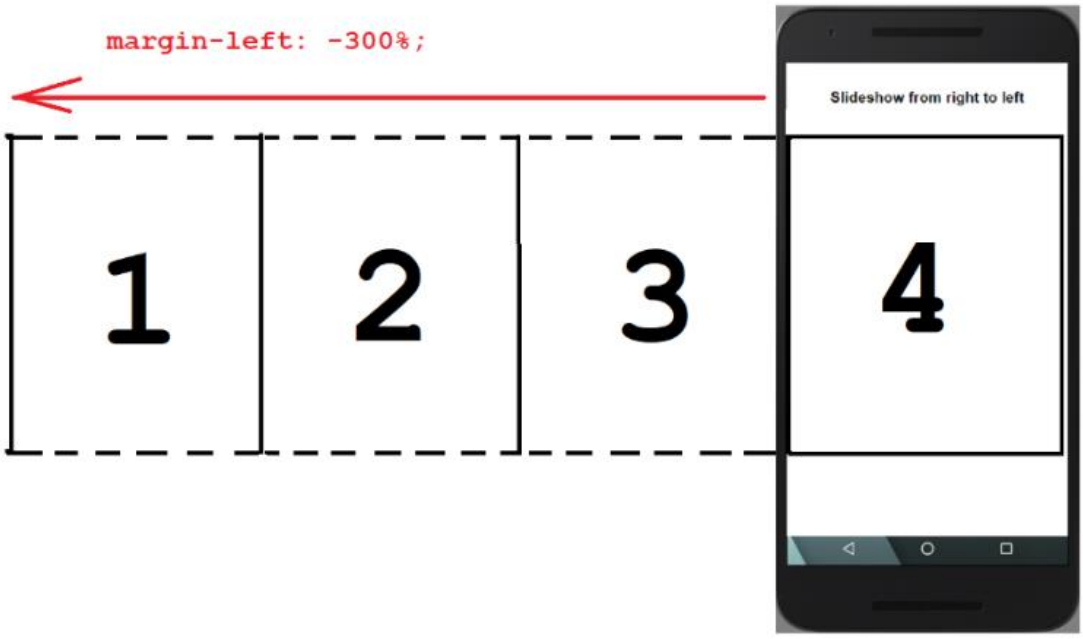
At the second time frame, the **margin-left** set to **-100%** to show the second image:



At the third time frame, the **margin-left** set to **-200%** to show the third image:



At the fourth time frame, the **margin-left** set to **-300%** to show the fourth image:



CSS code

```
.slide-wrapper {
  width: 400%;
  height: 100%;
  animation: slideshow 12s infinite;
}
@keyframes slideshow {
  0% {margin-left: 0px;}
  35% {margin-left: -100%;}
  70% {margin-left: -200%;}
  100% {margin-left: -300%;}
}
```

If we run the slideshow, nothing is showing because we have not set each image into the **slide-wrapper** yet. Since we are planning to show four numbers, then we need to create four slide divisions

HTML code

```
<body>
    <div id="slideshow1">
          <div class="slide-wrapper">
               <div class="slide"><h1>1</h1></div>
               <div class="slide"><h1>2</h1></div>
               <div class="slide"><h1>3</h1></div>
               <div class="slide"><h1>4</h1></div>
          </div> <!–end of slide-wrapper -->
    </div> <!–end of slideshow1 -->
</body>
```

Since all four **slide** divisions is set inside the **slide-wrapper**, then each **slide** division width must be set to 25%, $\frac{100\%}{4} = 25\%$

Also, all four divisions will be in-line from left to right, for this, we can set all four divisions to float left:

CSS code

```
.slide {
  float: left;
  height: 100%;
  width: 25%;}
```

We can also add some CSS to all **h1** within **slide** class:

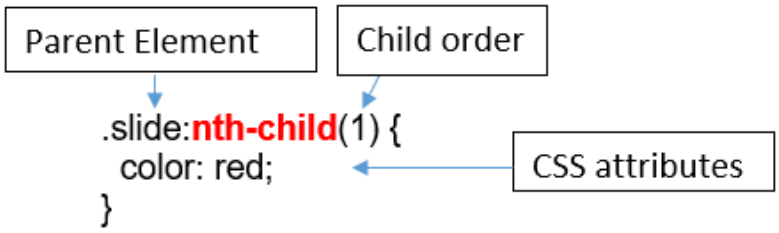
CSS code

```
.slide h1{
  text-align: center;
  font-size: 10em;}
```

Now, we can run the slideshow and see the number displaying one at the time.

The next features to add to each slide is the background color. There is different way we can set different background color to each slide, for example, we can use different class name to each slide division, or add the CSS code in the HTML code using **style="background-color: yellow;"**

For this activity, we are going to use the **nth-child** selector in CSS to set different background color to each **slide** division. The **nth-child** selector is a structural pseudo-class, meaning it is used to style content based on its relationship with the parent element. For our program, the parent element will be **slide** because **slide** has four divisions of the same class name. Then, each **slide** division, according to line order, will be the **nth-child** of the **slide** element. The syntax of the **nth-child** is:



For example, if we are setting the background color for the first slide to lightblue, then we can write the code as:

```
.slide:nth-child(1) {
  background-color:lightblue;
}
```

Then, to set different background color to each four slide elements:

CSS code

```
.slide:nth-child(1) {background-color:lightblue; }
.slide:nth-child(2) {background-color: pink;}
.slide:nth-child(3) {background-color: lightgreen;}
.slide:nth-child(4) {background-color: orange;}
```

Now, we can run the slideshow and see the number of each slide changes with different background color.

If we want each number of the slideshow to rest before it shifts to the next number, we can adjust the previous animation's time frame, **@keyframes** slideshow, to rest

by 10% of the total time frame at each slide, and 20% of the total time frame as the transition time between slide:

CSS code

```
@keyframes slideshow {
  0% {margin-left: 0px;}
  10% {margin-left: 0px;}
  30% {margin-left: -100%;}
  40% {margin-left: -100%;}
  60% {margin-left: -200%;}
  70% {margin-left: -200%;}
  90% {margin-left: -300%;}
  100% {margin-left: -300%;}
}
```

Run the slideshow and see how the number rest for 1.2 seconds, 10% of the total time frame 12 seconds, before it moves to the next number.

The other features that we can add to the animation is to see slideshow images to backward to the beginning to the slideshow when it reaches the last slide. For this, we can adjust the time frame to up 85% for the last slide. This will allow the slideshow to have the rest 86% to 100% to backward to the beginning to the slideshow. Therefore, the rest time at each slide will be 10% of the total time frame, the transition time between slides will be 15% of the total time frame, and the backward time will be 15% of the total time frame, which is from 86% to 100%:

CSS code

```
@keyframes slideshow {
  0% {margin-left: 0px;}
  10% {margin-left: 0px;}
  25% {margin-left: -100%;}
  35% {margin-left: -100%;}
  50% {margin-left: -200%;}
  60% {margin-left: -200%;}
  75% {margin-left: -300%;}
  85% {margin-left: -300%;}
}
```

3.11.2. Creating a modal window using HTML and CSS

Modal windows are boxes that opens in front on the computer screen. They are widely used for advertisements, promotion code, item description, login/register forms, etc.

There are different techniques to create modal windows, one way to create a modal window is by using CSS's attributes as transition, opacity, pointer-event, and background gradient properties.

To create a modal window, we need to create open modal link first:

HTML code

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <meta charset="utf-8">
    <title>Modal Window</title>
     <link rel="stylesheet" href="modal.css"
     type="text/css">
  </head>
  <body>
     <a href="#openModal" class="linkModel">Open Modal
     Link</a>
  </body>
</html>
```

CSS code

```
.linkModel{
text-align:center;
background-color: orange;
font-size:  20px;
padding: 10px;
border-radius: 20px;
text-decoration:none; }
```

Run the program to see if the modal link looks like this:



From the HTML code, the code line:

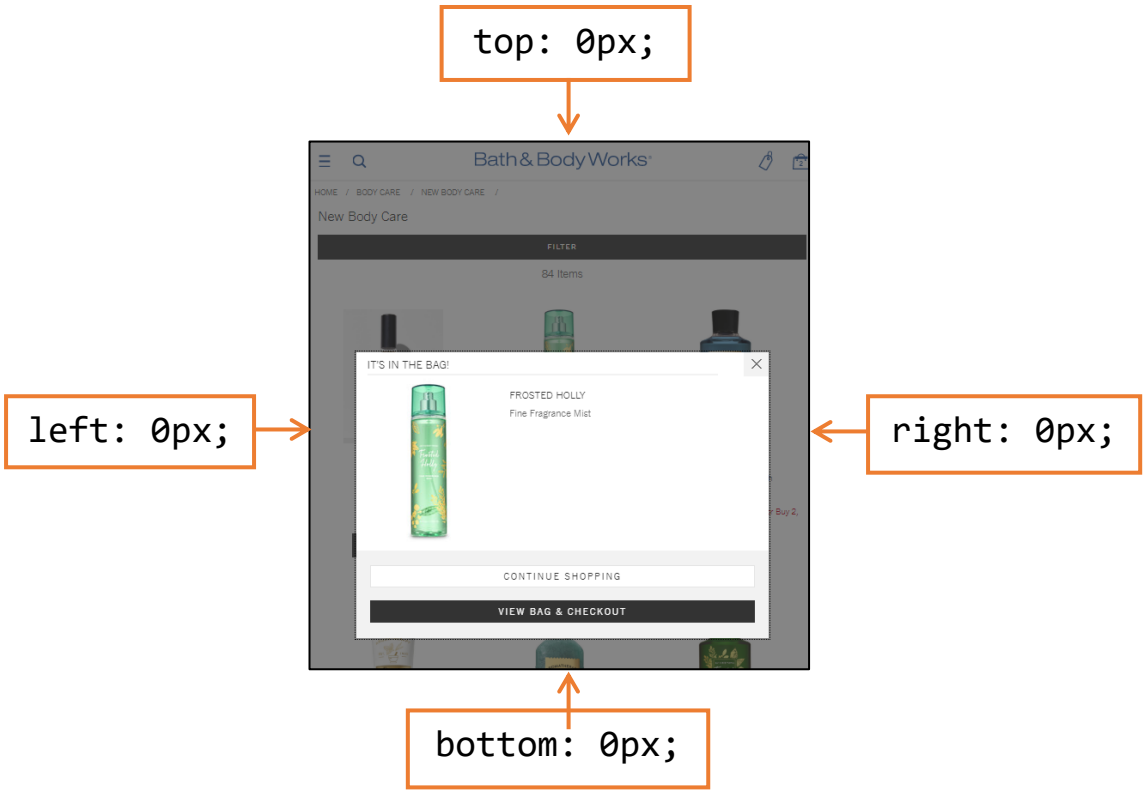
```
<a href="#openModal" class="linkModel">Open Modal
Link</a>
```

we can see that the modal is linked to an element **id** as "#openModal". Therefore, we have to create the modal element using element and we will **id** it as: id="openModal". This division we will also has a class name as: **class="modalWindow"**for css attributes.

HTML code

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <meta charset="utf-8">
    <title>Modal Window</title>
     <link rel="stylesheet" href="modal.css"
     type="text/css">
  </head>
  <body>
     <a href="#openModal" class="linkModel">Open Modal
     Link</a>
    <!-- Modal Window container -->
    <div id="openModal" class="modalWindow">
    </div>
  </body>
</html>
```

Since the modal window will cover the entire screen, we can set the top, right, left, and bottom to 0px. This setting means that there will be 0px of distance between the top, right, left, and bottom to the modal window.



Once the modal window is created, we have to apply transparency to the modal window. This property can be set by setting the window background to 80% of transparency using the **rgba** color property:

```
background-color: rgba(160,80,160,0.8);
```

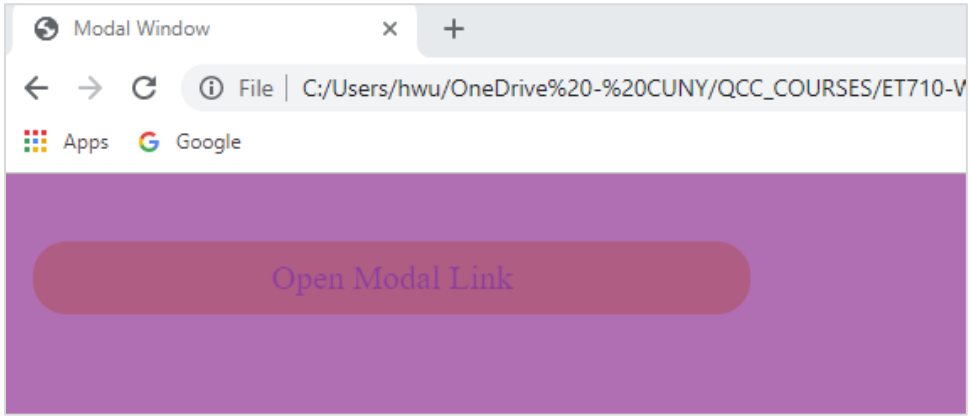
The CSS code should look as the following:

CSS code:

```
.linkModel{
width: 60%;
text-align:center; background-color: orange; font-size:
20px; padding: 10px;
border-radius: 20px;
}
.linkModel a{ text-decoration:none; }
```

```
.modalWindow {
position: fixed; top: 0;
right: 0;
bottom: 0;
left: 0;
background-color: rgba(160,80,160,0.8);
}
```

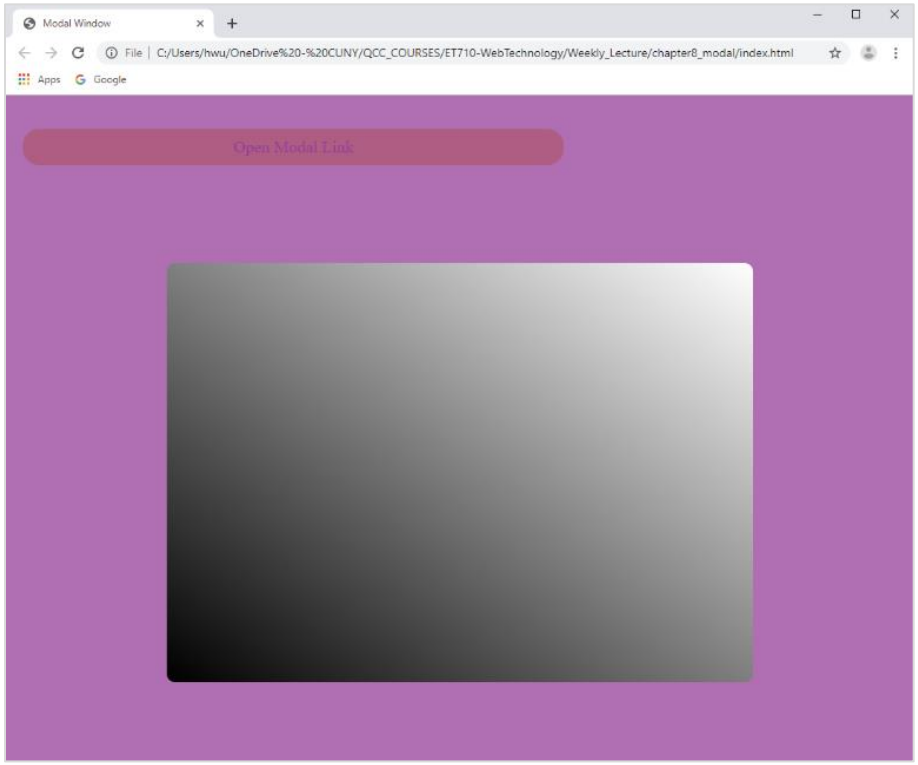
The internet browser should look as:



Having the background of the modal window set, we can add a message window on top of the modal window using a with class name **msgText.** This message window is used to display the advertisement of the modal. Since the message window will display in-front of the modal window, therefore this division will have **position: relative**. To make the message window more colorful and organized, we can add linear-gradient background to it, set the width to 80%, the height to border- radius, and 40% of top margin:

```
.msgText{
position: relative;
width: 700px;
height: 500px;
border-radius: 10px;
margin: auto;
margin-top: 200px;
background: linear-gradient(to top right, black, white);
}
```

The internet browser should look as:

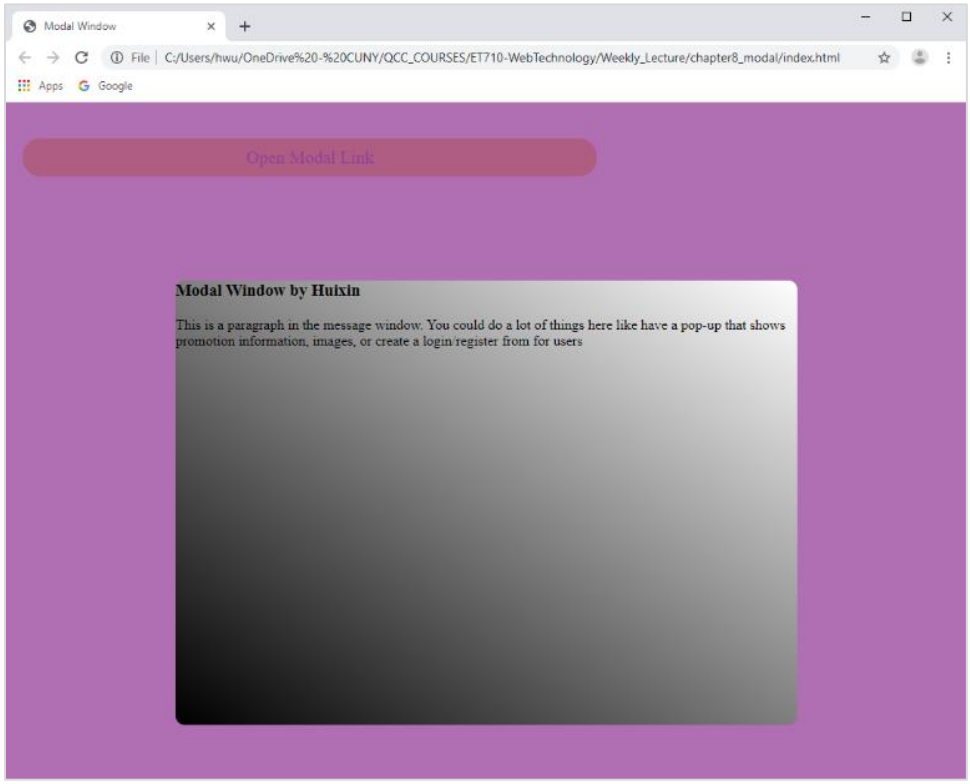


msgText is used as a message window. Therefore, we can add a title to this message window using **<h3>**and a text container using **<p>.**

HTML code

```
<body>
    <a href="#openModal" class="linkModel">Open Modal
    Link</a>
    <!-- Modal Window container -->
    <div id="openModal" class="modalWindow">
.     <div class="msgText">
        <h3>Modal Window by Huixin</h3>
        <p>This is a paragraph in the message window.
        You could do a lot of things here like have a
        pop-up that shows promotion information, images,
        or create a login/register from for users</p>
     </div>
    </div>
  </body>
```

Refreshing the internet browser, the modal window should look as:

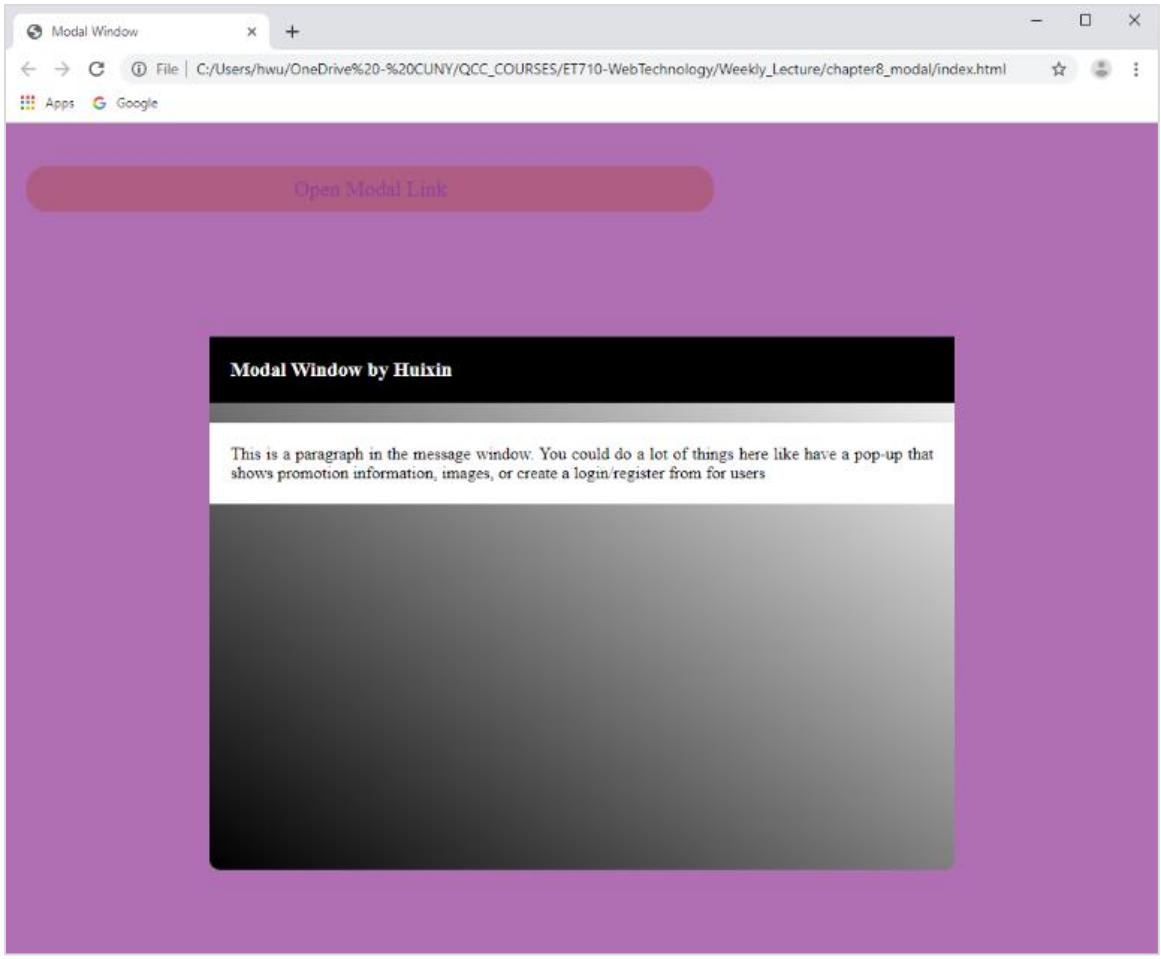


To make my message window more styling, in CSS, we add padding to **<h3>** and **<p>,**and align the text to justify in **<p>**. Remember that to call the **<h3>** and **<p>** without using a class name, we will need to call the division that contains them first and then the element, for example: **.msgText h3**

```
.msgText h3{
padding: 20px; background-color: black; color: white; }

.msgText p{
padding: 20px; text-align: justify; background-color:
white;}
```

Refreshing the internet browser, the modal window should look as:



We can also add an image to the message window:

HTML code

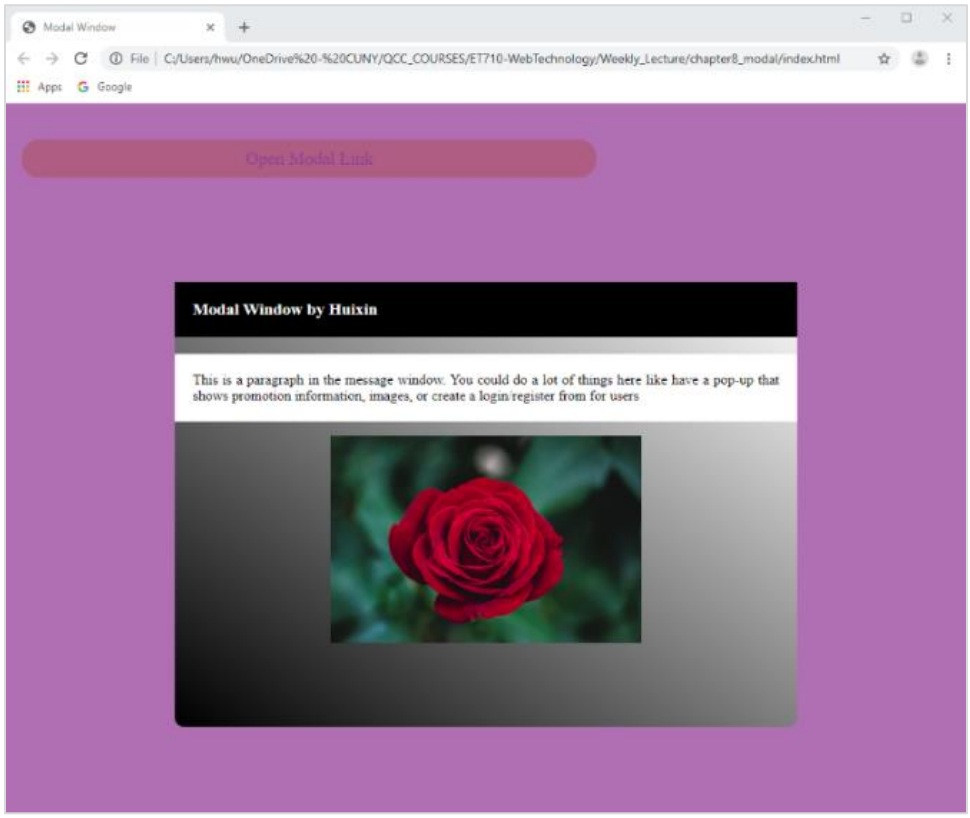
```
<body>
     <a href="#openModal" class="linkModel">Open Modal
     Link</a>
     <!-- Modal Window container -->
     <div id="openModal" class="modalWindow">
       <div class="msgText">
          <h3>Modal Window by Huixin</h3>
          <p>This is a paragraph in the message window.
          You could do a lot of things here like have a
          pop-up that shows promotion information, images,
          or create a login/register from for users</p>
```

```
        <img src="rose.jpg" alt="Image of a rose"
        title="Rose">
      </div>
    </div>
</body>
```

To style the image in the message window, we can make the width of the image to 50% of the message window width, and center the image by setting the margin-left or margin-right to 25%:

```
.msgText img{
  width: 50%;
  height: auto;
  margin-left: 25%;
}
```

Refreshing the internet browser, the modal window looks as:



The next thing to add to the message window **msgText** is a link to close the

modal window. For this, we can use an anchor **<a>** tag and link it to the top of the web app using: **href="#"**. We can name it as **class="modal-close"**for CSS attributes.

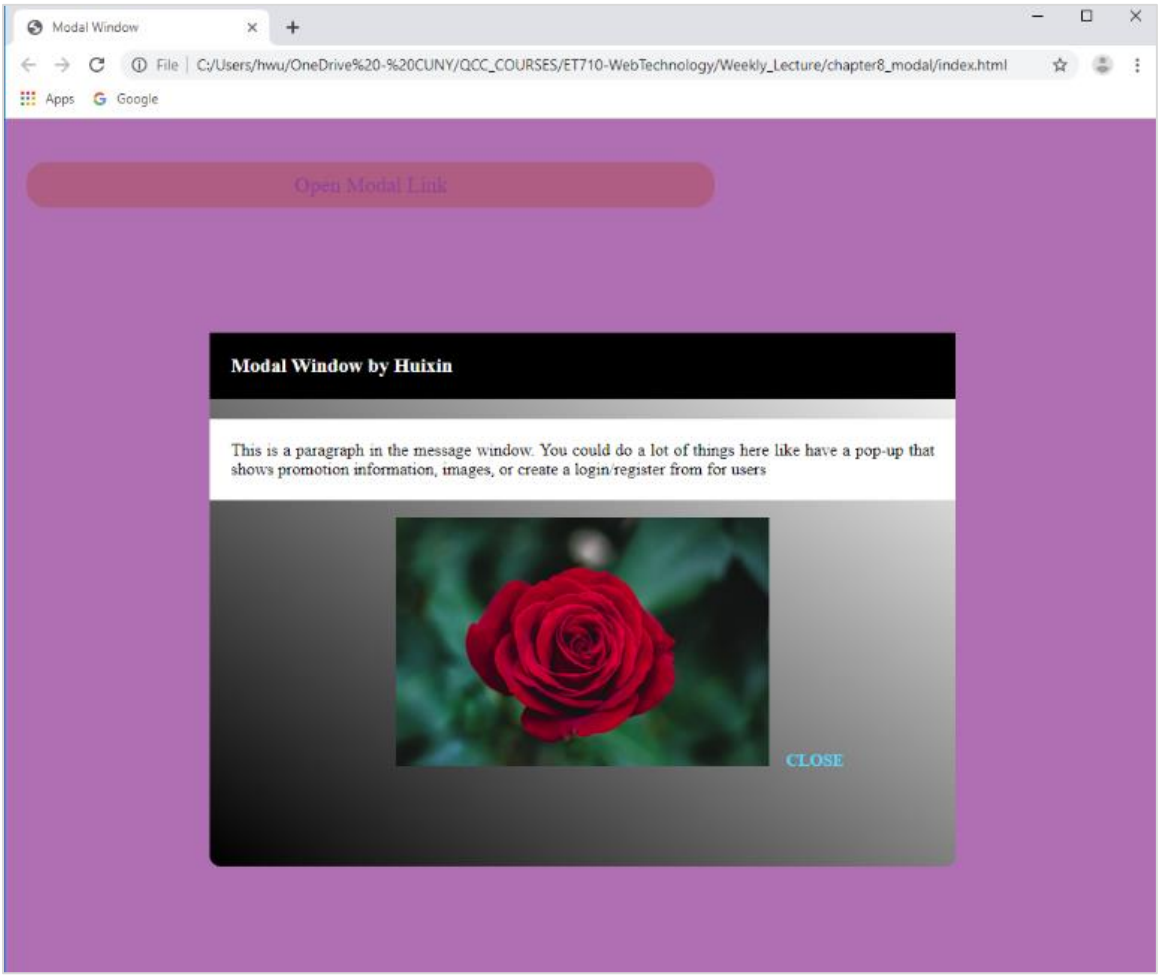
HTML code

```
<body>
    <a href="#openModal" class="linkModel">Open Modal
    Link</a>
    <!-- Modal Window container -->
    <div id="openModal" class="modalWindow">
      <div class="msgText">
         <h3>Modal Window by Huixin</h3>
         <p>This is a paragraph in the message window.
         You could do a lot of things here like have a
         pop-up that shows promotion information, images,
         or create a login/register from for users</p>
         <img src="rose.jpg" alt="Image of a rose"
         title="Rose">
         <a href="#" class="modal-close">CLOSE</a>
      </div>
    </div>
  </body>
</html>
```

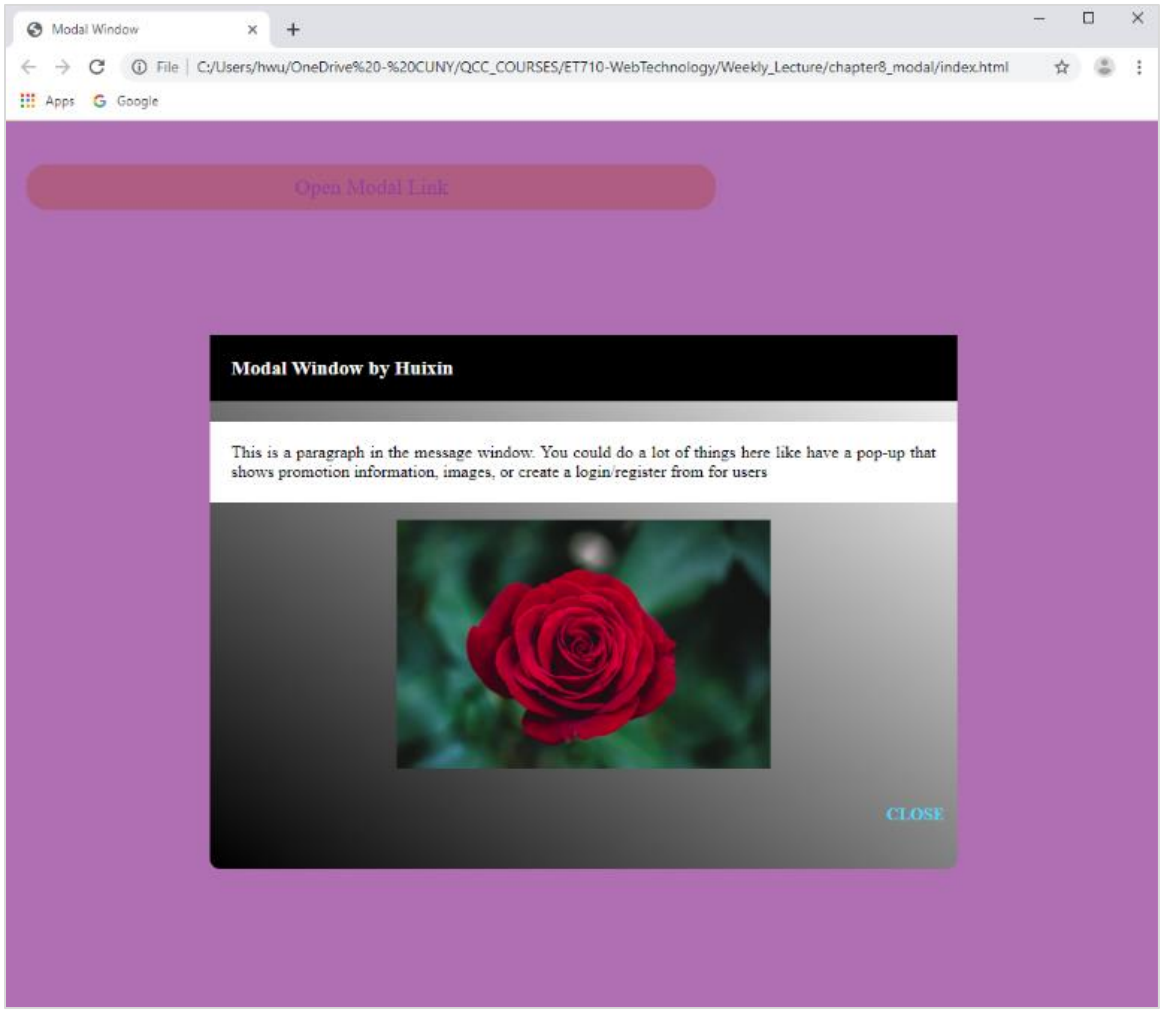
Now is CSS, we can set attributes to **.modal-close**such as width, the font color, padding, font-weight, text-align, and set the text-decoration to none:

```
.modal-close {
color: #00d9ff;
padding: 12px;
font-weight: bold;
text-align: center;
text-decoration: none; }
```

Refreshing the internet browser, the webpage looks as:



We can also add two break lines **
** in between the image and the CLOSE link to move the CLOSE link to the next line. If we want to have the CLOSE link in the right side, we can use **float:right;** attribute.



Once we have the modal window set, we can set the **opacity** attribute to it. The idea of using the **opacity** attribute is to make the modal window to invisible, **opacity:0;** , when the web app is loaded, then when the modal link is clicked, the modal window will become visible, **opacity: 1; .** To set this attribute, we add **opacity:0;** in the CSS of the modal window **.modalWindow**

```
.modalWindow {
position: fixed; top: 0;
right: 0;
bottom: 0;
left: 0;
background-color: rgba(160,80,160,0.8);
opacity: 0;
}
```

The other setting is to create a hover effect to the modal window to open to modal window using **opacity: 1;**

```
.modalWindow:hover{
  opacity: 1;
}
```

If we run the app, the message window will not close when we click on the CLOSE link. This happens because we did not set the **pointer-events**to **target** an element that has the CLOSE link. For this, we can include **pointer-events:none**; to the element that has the anchor tag **<a>** beneath it. In this activity, we target the modal window **.modalWindow** and when click on the CLOSE link, the model window will close, that is why we need to change **.modalWindow:hover** to **.modalWindow:target** and add a **pointer-events: auto;** which means that the any anchor **<a>**tag under **.modalWindow**will react to the pointer events. In other words, when we click on the CLOSE link, the anchor **<a>** tag will take us to the beginning to the web app as states with **href="#"**

```
.modalWindow {
position: fixed; top: 0;
right: 0;
bottom: 0;
left: 0;
background-color: rgba(160,80,160,0.8);
opacity: 0;
pointer-events: none;
}
.modalWindow:target{
  opacity: 1;
  pointer-events: auto;
}
```

We can also add some animation to our modal window, for example, we can make the modal window to **ease-in**when it opens.

```
.modalWindow:target{
  opacity: 1;
  pointer-events: auto;
  transition: ease-in 0.5s;
}
```

Bibliography

Duckett, J. (2016). *HTML and CSS Design and build websites.* Indianapolis: John Wiley and Sons Inc.

Duckett, J. (2016). *JavaScript and JQuery: interactive front-end developer.* Indianapolis: John Wiley and Sons Inc.

HTML, CSS, JavaScript, JQuery, and Bootstrap. (2017, June). Retrieved from w3schools: www.w3schools.com

Some material compiled from www.lynda.com (May 2018), www.w3schools.com (2020), www.coursera.org (2020)

© 2026. This OER resource is licensed under a Creative Commons license (BY-NC-SA). It may be freely shared and redistributed for non-commercial purposes, provided proper attribution is given and any derivatives are shared under the same license.

Unauthorized copying, reproduction, or distribution of this material, in whole or in part, without permission is not allowed.